Cognizant Flowsource

– Deployment Document

Contents

1. Objective 3
Backstage prerequisite 3
Create app using npx 3
Adding configuration files 3
Flowsource-Core Installer 4
2. Installer file structure 4
3. Prerequisites 4
4. Configuration Details 5
5. Installation Steps 20
6. GitHub OAuth Configuration 21
7. Configure GitHub Authentication 21
8. Azure AD Authentication provider 24
9. AWS Cognito Authentication provider 29
10. Keycloak Authentication provider 36
11. Flowsource and Authentication provider role/group mapping 47
12. Flowsource Data Engine installation 49
13. FlowSource CSS changes 49
14. Changes required to implement Proxied Authentication using AWS ALB with Azure AD provider 50
15. Changes required to implement Proxied Authentication using Oauth2 Proxy (with GitHub provider / Azure AD provider) 50
16. Registering Templates with Flowsource 51
17. Proxied Authentication using GCP-IAP with Microsoft Entra ID (Azure Active Directory) 52
18. Rest API to add/ delete rows in role_mappings and email_to_role_mappings in auth database. 55
19. AWS Master pipeline template 57
20. GCP Master Pipeline Template 71
21. Azure DevOps Discovery 73

1. Objective

This document contains all the relevant details & steps to deploy Cognizant Flowsource. Please don't share this document to others and use it only for your reference.

Backstage prerequisite

Ensure that the following software is installed on the system from where deployment would be performed:

- Node js (*preferrable version is 22.X.X*)
- Yarn (*preferrable version is 4.X.X*)
- Python (*preferrable version is 3.x.x*)

Create app using npx

Flowsource-Core packages only include plugins. However, they do not provide pre-packaged backstage source files. To generate these source files, execute the following 'npx' command. The command below creates the default backstage source files while skipping the installation process.

```
npx @backstage/create-app@0.5.25 --path ./ --skip-install
```

Note: Before executing the command, please ensure to back up the app-config.yaml file. Replace the file once the app is created.

FlowSource Set up with Authentication (without Plugins)

For setting up a basic FlowSource instance (Backstage + FlowSource theme) with authentication, follow the instructions provided at –

```
Flowsource_Package_<release_version>.zip/FlowSourceInstaller/FlowsourceSetupDoc/Readme.md
```

Adding configuration files

After the npx command finishes execution and generates the backstage source files, copy the files from */configuration* directory to the current working directory.

Flowsource-Core Installer

Flowsource installer is based on Terraform & Helm chart to install the required resources such as service, deployment, secrets, configmaps, ingress controller or load balancer. More details about the installer is present at Flowsource-Package_<version>.zip/FlowSourceInstaller/ReadMe.MD

2. Installer file structure

The Cognizant Flowsource installer package contains the following contents:

Installer file structure	
flowsource folder	This folder contains helm template and other supporting configuration files
flowsource.tar	This tar file contains the containerized image of the current Flowsource application
Install (.sh/.bat)	This install script helps to install the Flowsource platform

3. Prerequisites

Please find the below prerequisites to be present in the machine used for installation.

Prerequisites Details	
Docker	It is mandatory
Kubectl	It is mandatory
Helm	It is mandatory
Terraform	It is mandatory
Deployment Platform	
AWS	• S3, ECR and EKS access is needed • Please create a repo in the ECR and a bucket in S3 • Configure aws in the terminal using *aws configure* command • Create an EKS cluster and a namespace in it and setup the EKS Cluster

Azure	• ACR and AKS access is needed • Please create an ACR Repo and AKS Cluster • Configure the azure login using *az login* command and setup the AKS cluster config
Service	
There is an option to select between ingress controller and loadbalancer during the Helm installation to Kubernetes. This can be setup by setting yes/no to the *useIngress* field in *FlowSourceInstaller/backstage/values.yaml.*	
Ingress	• The yes option chooses ingress controller. • For this, the prerequisite is to setup Ingress Controller in the respective Kubernetes clusters
Load balancer	• The no option chooses load balancer setup that each Kubernetes cluster supports
Postgres	
There is an option to select between External Postgres Instance and Internal Postgres Instance. This can be setup by setting yes/no to the *useInternalPostgres* field in *FlowSourceInstaller/backstage/values.yaml.*	
Internal Postgres	• The yes option chooses internal postgres instance • Values must be updated to *POSTGRES_USER* and *POSTGRES_PASSWORD* in the *flowsourceCommonPostgresConfig* section of the *values.yaml* file • Postgres deployment takes place as part of flowsource deployment • Create a storageclass with name *standard* for the PersistentVolumeClaim to access postgres data in the respective Kubernetes cluster
External Postgres	• The no option chooses external postgres instance • Values must be updated to *POSTGRES_SERVICE_HOST* and *POSTGRES_SERVICE_PORT* in the *flowsourceExternalPostgresConfig* section along with the *flowsourceCommonPostgresConfig* section (user and password)

4. Configuration Details

Below are the configurations that needs to be updated "***FlowSourceInstaller/terraform-scripts/values-files/values.yaml*** " before running the Flowsource installer.

Values.yaml	
replicaCount	Defines the replicas of the deployment of both Flowsource and postgres deployment
namespace	Defines the namespace in the Kubernetes cluster where the deployment is done
useInternalPostgres	Defines if internal postgres will be used. Refer to point 7 in Prerequisites
flowsourceDeploy.image	This will be autoupdated with the Flowsource image value during the installation
flowsource.flowsourceDeploy.serviceAccount.name	If flowsourceSecrets.useSecretProvider is set to true, specify the service account name that will be used in deployment.yaml. If not provided, Kubernetes defaults to the namespace's default service account.
flowsource.flowsourceDeploy.serviceAccount.annotations	If flowsourceSecrets.useSecretProvider is set to true, define a key-value pair mapping the role ARN key to its respective value. AWS: eks.amazonaws.com/role-arn: ROLE_NAME The Role ARN must have access to Secrets Manager. GCP: iam.gke.io/gcp-service-account: ProjectID.iam.gserviceaccount.com
flowsourceConfig	
This flowsource config file defines the config map values needed for flowsource. Fields of this section is described below (* - Fields to be updated manually)	
NODE_OPTIONS	Defines the node options. This needs no change
SONARQUBE_URL*	Defines the sonarqube URL
CHECKMARX_BASE_URL*	Defines the Checkmarx URL

CHECKMARX_AUTH_URL*	Defines the checkmarx authentication URL
DEVOPS_GURU_FILTER_FROM_TIME	This value is set to 0. Needs no change
ChatBotJsonsFolderPath	This path is for accessing chatbot and needs no change
FLOWSOURCE_BASE_URL	This is the base URL of the running container / application. When running locally, with separate frontend and backend, app.baseUrl and backend.cors.origin should be set to http://localhost:3000. backend.baseUrl should be set to http://localhost:7007. When running in cloud (with frontend and backend together), this value should have the base URL of Flowsource.
DATADOG_URL*	Defines the datadog URL part of csp.frame-src. If further details to be added to frame-src, it can be added directly in *app-config.yaml*
JIRA_URL	Defines the Jira URL
S3_BUCKET_NAME*	Defines the s3 bucket name where the techdocs will be stored. Need read write permission to this s3 bucket
FLOWSOURCE_VERSION	Defines the flowsource version and will be updated during the installation
JIRA_USER_EMAIL	Defines user email for Jira
JIRA_BASE_URL	Jira base url for the rest api
JIRA_TARGET	Jira URL
TECHDOCS_STORAGE_PROVIDER_TYPE	Defines Techdocs storage provide type
TECHDOCS_STORAGE_CONTAINER_NAME	Defines Techdocs storage container name
CHATBOT_ENABLED	Set it to ‘true’ to enable chatbot

PRISMA_API_URL	Prisma cloud url for api calls
AZURE_DEVOPS_BASEURL	Azure Dev Ops base url
CHATBOT_URL	Chat bot component url
JENKINS_BASE_URL	Jenkins server base url for api calls
MORPHEUS_HOST_URL	IP/URL of the Morpheus server instance
GRANT_TYPE	This value determines the type of authentication mechanism while interacting with Morpheus server. Example – 'password'
SCOPE	Nature of interaction with Morpheus server – whether api calls can result in a write action or just a read only possible. Example – 'write'
FLOWSOURCE_USER_ID	This is required Morpheus api calls. The user id to identify the changes made in Morpheus server by FlowSource API calls.
MORPHEUS_REJ_UN_AUTH	A flag to enable/disable TLS, while communicating with Morpheus Server over HTTPS
MORPHEUS_ORDER_SUBMISSION_ENABLED	Default value will be false. Determines whether Morpheus service-catalog order can be placed using api calls.
MORPHEUS_INPUT_FIELD_DATA_LEN	default will be 250 chars. Number of characters allowed in service-catalog input field
CICD_GITHUB_ACTIONS_BASEURL	Github base API url which will be used for querying github actions data over rest.
GOOGLE_TYPE	GCP account type for cicd gcp plugin to connect. Example – 'service_account'
GOOGLE_PROJECT_ID	GCP project id from which cicd data would be queried
GOOGLE_CLIENT_ID	A unique id specific to flowsource as a client application which will query Google APIs.

GOOGLE_AUTH_URI	GCP Admin to provide as part of API Authentication details.
GOOGLE_TOKEN_URI	GCP Admin to provide as part of API Authentication details.
GOOGLE_AUTH_PROVIDER_X509_CERT_URL	GCP Admin to provide as part of API Authentication details.
GOOGLE_CLIENT_X509_CERT_URL	GCP Admin to provide as part of API Authentication details.
GOOGLE_UNIVERSE_DOMAIN	GCP Admin to provide as part of API Authentication details.
SOURCE_CODE_PROVIDER	GCP Admin to provide as part of API Authentication details.
GITHUB_ENABLED	Used for flagging GCP CICD plugin to determine if github is used as SCM
AWS_ALB_REGION	This parameter would be used by AWS ALB for proxy authentication.
AUTH_IDLE_TIMEOUTMINUTES	Sets the idle time (in minutes) after which the user will be logged out.
AUTH_PROMPT_BEFORE_IDLESECONDS	Determines the time (in seconds) prior to idle state when a prompt will appear. This must be less than the value of idleTimeoutMinutes.
TWISTLOCK_API_HOST	Twist lock url to be obtained from Prisma cloud portal for a given tenant
INSIGHTS_REDSHIFT_DATABASE	Redshift Database name for getting DORA Metrics data
INSIGHTS_REDSHIFT_SCHEMA	Redshift Database Schema for getting DORA Metrics data
INSIGHTS_REDSHIFT_REGION	Redshift Database Region for getting DORA Metrics data
INSIGHTS_REDSHIFT_SERVERLESS_WORKGROUP_NAME	Redshift Serverless Workgroup name for getting DORA Metrics data. This is specific only for Serverless Redshift DB
INSIGHTS_REDSHIFT_SERVERLESS_SECRET_ARN	Redshift Serverless Secret ARN for getting DORA Metrics data. This is specific only for Serverless Redshift DB

INSIGHTS_REDSHIFT_DB_USER	Redshift Database username for getting DORA Metrics data. This is specific only for Redshift cluster-based DB
SOURCE_CODE_PROVIDER	The SCM provider for GCP platform integration. Example – 'github'
INSIGHTS_REDSHIFT_CLUSTER_IDENTIFIER	Redshift Database cluster identifier for getting DORA Metrics data. This is specific only for Redshift cluster-based DB
DASHBOARD_DOMAIN	Domain part of Quicksight url – so that iFrames are allowed as part of context security policy
GOOGLE_BUCKET_NAME	Storage bucket in GCP for storing tech doc related files
GOOGLE_PROJECT_ID	GCP project id, under which the storage bucket is referred for storing tech docs related files
AZURE_PIPELINE_VARIABLE_GROUP	Variable groups store values and secrets that you can pass into a YAML pipeline or make available across multiple pipelines in a project. Ex: master-pipeline-variable-group
AZURE_MASTER_PIPELINE_NAME	Used as part of springboot3-azure-ci-pipeline-template which creates a master pipeline in azure for the CI pipelines
DASHBOARD_EMBED_URL	
GCP_IAP_AUDIENCE	GCP platform's identity aware proxy authentication against project/service id
INSIGHTS_CLOUDPROVIDER	Defines the Cloud Provider for Dora Metrics plugin
CATALOG_INFO_CLOUD_PROVIDER	Defines the Cloud Provider where the storage to push the catalog data is hosted.
INTEGRATIONS_GITHUB_APP_APP_ID	A unique application id for registering it with github
INTEGRATIONS_GITHUB_APP_CLIENT_ID	A unique id for Flowsource to register itself with github for authentication and authorization
GIT_TRIGGER_ID	Unique id representing the github triggers which would execute the builds
CATALOG_S3_REGION	AWS s3 region where catalog items would be stored

CATALOG_S3_BUCKET_NAME	AWS S3 bucket name where the catalog items would be stored
CATALOG_S3_UPLOAD_PATH	AWS S3 upload path to which catalog items would be uploaded
JIRA_STORY_POINTS_FIELD	Number of days to look back for fetching data and displaying in the frontend
K8S_EKS_CLUSTER_NAME	Kubernetes cluster name to which the Kubernetes plugin would connect to get information of the catalog component
K8S_EKS_CLUSTER_ENDPOINT	Kubernetes cluster endpoint to which the Kubernetes plugin would connect to get information of the catalog component
K8S_AWS_ACCOUNT_ID	Kubernetes cluster's AWS account id to which the Kubernetes plugin would connect to get information of the catalog component
K8S_EKS_CA_DATA	Cryptographic secret data for Kubernetes plugin to connect with Kubernetes cluster
VERACODE_API_ID	Unique app id to for verracode to identify Flowsource
DYNATRACE_CLIENTID	Unique id for flowsource to connect to Dynatrace over rest api
DYNATRACE_RESOURCE	A unique resource id which represents collection of resources to which the client has been granted access to specific scopes.
DYNATRACE_ENVIRONMENTID	A unique id representing the Dynatrace account to which Flowsource needs to connect for api calls.
AWS_MASTER_PIPELINE	Master pipeline name in AWS Codepipeline
CODEPIPELINE_GITHUB_ORG_CONNECTION_ARN	The connection ARN in AWS CodePipeline for the GitHub org used.
AZURE_DEVOPS_REPOSITORY	The repository name. Wildcards are supported as show on the examples above. If not set, all repositories will be searched.
AZURE_DEVOPS_ORG	Your Organization slug (or Collection for on-premise users).
AZURE_DEVOPS_PROJECT	Your project slug. Using '*' will search all projects. For a project name containing spaces, use both single and double quotes as in project: '"My Project Name"'.

INSIGHTS_GCS_DATASET_NAME	The GCP Biquery DataSet Name for getting DORA Metrics data
INSIGHTS_GCS_PROJECT_ID	GCP project id under which the Data Set Name is referred for getting DORA metrics data
CATALOG_GCS_BUCKET_NAME	Storage bucket name in GCP where the catalog items would be stored
CATALOG_GCS_UPLOAD_PATH	Path to which catalog items would be uploaded in the GCP Storage bucket
CATALOG_GCS_PROJECT_ID	project id under which the Storage bucket is referred to upload catalog items
flowsourceExternalPostgresConfig	
This contains values that will be updated if external postgres instance is used	
POSTGRES_SERVICE_HOST	The URL of the external database instance
POSTGRES_SERVICE_PORT	Port to access the external database
CODEPIPELINE_REGION	Defines the aws region for Aws Codecommit service or plugin
BITBUCKET_CLIENTID	Unique ID created for application while performing authentication & Authirization based on OAuth2.0 for bitbucket
BITBUCKET_API_URL	Base url used by plugins to form the complete API urls
BITBUCKET_OAUTH_URL	Link to OAuth server URL for plugin to send authentication and authorization request
APPDYNAMICS_BASEURL	Base url used by plugins to form the complete API urls
APPDYNAMICS_CLIENTID	Unique ID created for application while performing authentication & Authirization based on OAuth2.0 for AppDynamics
SONARQUBE_VERSION	SonarQube server version
PLAYWRIGHT_S3_ACCESS_KEY	AWS access key used by Playwright plugin for accessing s3 bucket, where test suite details are stored
PLAYWRIGHT_S3_ROLE_ARN	AWS role ARN used by Playwright plugin for accessing s3 bucket, where test suite details are stored

PLAYWRIGHT_S3_REGION	AWS region used by Playwright plugin for accessing s3 bucket, where test suite details are stored
PLAYWRIGHT_S3_BUCKET_NAME	AWS S3 bucket name used by Playwright plugin where test suite details are stored.
BITBUCKET_CLOUD_APP_USERNAME	User name for accessing bitbucket cloud using api
ZEPHYR_BASE_URL	Base URL for making api calls to Zephyr plugin.
DATADOG_RUM_APPLICATION_ID	Application ID which has to be registered for RUM in Datadog
DATADOG_RUM_SITE	Site address of the application which has to be registered for RUM in Datadog
DATADOG_RUM_ENV	ENV variables of application which has to be registered for RUM in Datadog
SONARQUBE_PROJECT_URL	Sonarqube project URL
FINOPS_CLOUDABILITY_REPORT_BASE_URL	Base url for api calls to cloudability
SELENIUM_S3_ACCESS_KEY	AWS access key for accessing S3 buckets by selenium plugin
SELENIUM_S3_ROLE_ARN	AWS role arn for accessing S3 buckets by selenium plugin
SELENIUM_S3_REGION	AWS S3 bucket region used by selenium plugin
SELENIUM_S3_BUCKET_NAME	AWS S3 bucket name used by selenium plugin
flowsourceCommonPostgresConfig	
This contains common values if either internal or external postgres is used.	
POSTGRES_PASSWORD	Defines the postgres password of either internal or external postgres instance
POSTGRES_USER	Define the postgres username of either internal or external postgres instance
flowsourceSecrets	
This contains fields that store secrets that are configured to Kubernetes secrets	
GITHUB_TOKEN	Defines the github private token to access the github

S3_AWS_ACCESS_KEY	Defines the aws access key to access s3
S3_AWS_SECRET_KEY	Defines the aws secret key to access s3
S3_AWS_REGION	Defines the aws region
WELL_ARCHITECTED_AWS_ACCESS_KEY	Defines the aws access key to well architected plugin data
WELL_ARCHITECTED_AWS_SECRET_KEY	Defines the aws secret key to access well architected plugin data
AUTH_GITHUB_CLIENT_ID	Defines the github client id
AUTH_GITHUB_CLIENT_SECRET	Defines the github client secret
AUTH_ATLASSIAN_CLIENT_ID	Defines the Atlassian client id for Jira plugin
AUTH_ATLASSIAN_CLIENT_SECRET	Defines the Atlassian client secret for Jira plugin
JIRA_TOKEN	Define the Jira token for Jira plugin
SONARQUBE_TOKEN	Defines the sonarqube token
CHECKMARX_CLIENT_ID	Defines the checkmarx client id for checkmarx plugin
CHECKMARX_CLIENT_SECRET	Defines the checkmarx client secret for checkmarx plugin
CHECKMARX_USERNAME	Defines the checkmarx username for checkmarx plugin
CHECKMARX_PASSWORD	Defines the checkmarx password for checkmarx plugin

CHECKMARX_GRANT_TYPE	Defines the checkmarx grant type for checkmarx plugin
AZURE_DEVOPS_TOKEN	Defines the azure devops token for the Azure Devops plugin
AZURE_DEVOPS_ORGANIZATION	Defines the azure devops organization detail for the Azure Devops plugin
DEVOPS_GURU_AWS_ACCESS_KEY_ID	Defines the aws access key for devops guru data
DEVOPS_GURU_AWS_SECRET_ACCESS_KEY	Defines the aws access key for devops guru data
BACKEND_AUTH	Defines for Service-to-Service auth string. It can be generated using the below command node -p 'require("crypto").randomBytes(24).toString("base64")'
JIRA_ACCESS_KEY	Defines the access key for Jira
DATADOG_API_KEY	Defines datadog API Key
DATADOG_APP_KEY	Defines datadog APP Key
TECHDOCS_AZURE_BLOB_STORAGE_ACCOUNT_NAME	Defines Techdocs blob storage account name
TECHDOCS_AZURE_BLOB_STORAGE_ACCOUNT_KEY	Defines Techdocs blob storage account key
PRISMA_ACCESS_ID	Prisma cloud Access ID
PRISMA_ACCESS_KEY	Prisma cloud secret key
BACKEND_SECRET	Secret for front end to back end authentication

JENKINS_USERNAME	Jenkins server username
JENKINS_API_KEY	Jenkins server api key
MORPHEUS_CLIENT_ID	Morpheus client id
MORPHEUS_PASSWORD	The Morpheus server instance's service user's password used for api calls authentication
MORPHEUS_USERNAME	The Morpheus server instance's service user used for api calls authentication
CODEPIPELINE_ACCESS_KEY	Defines the access key for Aws Codecommit service or plugin
CODEPIPELINE_SECRET	Defines the secret access key for Aws Codecommit service or plugin
CODEPIPELINE_ROLE_ARN	Unique Identifier for AWS role which should be defined with IAM permissions for accessing AWS code commit.
GITHUB_ORGANIZATION	Used for GITHUB as an auth provider. The github organization under which the oAuth app for FlowSource is configured.
AZURE_CLIENT_ID:	Unique id value assigned to client application (FlowSource) while configuring oAuth app in Azure AD
AZURE_CLIENT_SECRET:	Secret token for unique id value assigned to client application (FlowSource) while configuring oAuth app in Azure AD
AZURE_TENANT_ID:	Unique id assigned for an Organization in Microsoft Azure Active Directory
GOOGLE_PRIVATE_KEY_ID	In GCP, this is a unique id for a service account's private key.
GOOGLE_PRIVATE_KEY	A secret key in GCP, similar to user password used for authentication from a client over rest.
GOOGLE_CLIENT_EMAIL	GCP service account's email id, which will be used for querying GCP api's over rest.

GIT_TOKEN	A personal access token for github repo by which GCP code-build pipeline fetches the commit metadata from Github api's.
FINOPS_CLOUDABILITY_TOKEN	The access token used for api calls authentication to the cloudability server
INSIGHTS_REDSHIFT_AWS_ACCESS_KEY	Defines the aws access key to access Redshift
INSIGHTS_REDSHIFT_AWS_SECRET_KEY	Defines the aws secret key to access Redshift
GOOGLE_APPLICATION_CREDENTIALS	GCP platform credentials for interacting with storage bucket for storing and retrieving tech docs
CHATBOT_TOKEN	API authentication token for connecting chatbot component
ROLE_RESTAPI_APIKEY	API authentication token for accessing rest api to create and delete role_mappings and email_to_role_mappings tables in auth database.
AUTH_SESSION_SECRET	The auth session secret is generated using the below command. node -p 'require("crypto").randomBytes(24).toString("base64")' It is used by the OIDC provider Cognito and Keycloak
AUTH_AWS_COGNITO_METADATA_URL	It is the Open ID Configuration URL for Cognito that is created under Cognito User Pool
AUTH_AWS_COGNITO_TOKEN_URL	Open the Open ID configuration URL from the previous step in a browser and copy the token_endpoint URL and update it in the configuration. This endpoint is used to generate the OIDC token
AUTH_AWS_COGNITO_AUTHORIZATION_URL	Open the Open ID configuration URL from the previous step in a browser and copy the authorization_endpoint URL and update it in the configuration. It is used to authenticate the user
AUTH_AWS_COGNITO_CLIENT_ID	Client ID generated in under User Pools->Integration in AWS Cognito App.

AUTH_AWS_COGNITO_CLIENT_SECRET	Client secret generated in under User Pools->Integration in AWS Cognito App.
KEYCLOAK_METADATA_URL	It is the Open ID Configuration URL for Keycloal that is created during the realm creation under realm settings in Keycloak admin
KEYCLOAK_TOKEN_URL	Open the Open ID configuration URL from the previous step in a browser and copy the token_endpoint URL and update it in the configuration. This endpoint is used to generate the token
KEYCLOAK_AUTHORIZATION_URL	Open the Open ID configuration URL from the previous step in a browser and copy the authorization_endpoint URL and update it in the configuration. It is used to authenticate the user
KEYCLOAK_CLIENT_ID	Client ID generated during the keycloak client creation process in the admin console
KEYCLOAK_CLIENT_SECRET	Client secret generated during the keycloak client creation process in the admin console
INSIGHTS_AZURE_DB_HOST	Defines the host of the Azure Cosmos DB for Postgresql
INSIGHTS_AZURE_DB_PORT	Defines the port of the Azure Cosmos DB for Postgresql
INSIGHTS_AZURE_DB_USER	Defines the user of the Azure Cosmos DB for Postgresql
INSIGHTS_AZURE_DB_PASSWORD	Defines the password of the Azure Cosmos DB for Postgresql
INSIGHTS_AZURE_DB_DATABASE	Defines the database name of the Azure Cosmos DB for Postgresql
INSIGHTS_AZURE_DB_SCHEMA	Defines the schema of the Azure Cosmos DB for Postgresql
CATALOG_STORAGE_ACCOUNT_NAME	Defines the Azure Storage Account name

CATALOG_BLOB_CONTAINER_NAME	Defines the Azure Storage Blob container name
CATALOG_BLOB_FOLDER_NAME	Defines the Azure Storage Blob folder name
CATALOG_AZURE_CLIENT_ID	Defines the Client Id of the Service Principal to access Azure Storage Account
CATALOG_AZURE_TENANT_ID	Defines the Tenant Id of the Service Principal to access Azure Storage Account
CATALOG_AZURE_SECRET_KEY	Defines the Secret Key of the Service Principal to access Azure Storage Account
INTEGRATIONS_GITHUB_APP_CLIENT_SECRET	Defines a secret for flowsource to securely interact with Github
INTEGRATIONS_GITHUB_APP_PRIVATE_KEY	Cryptographic key used by github to securely authenticate Flowsource
INTEGRATIONS_GITHUB_APP_WEBHOOK_SECRET	Github app webhook secret – defined to verify incoming webhook payloads from Flowsource
CATALOG_S3_SECRET_ACCESS_KEY	Secret key to interact with AWS Managed services
K8S_AWS_ACCESS_KEY_ID	Access key to interact with AWS Managed services
K8S_AWS_SECRET_ACCESS_KEY	Secret key to interact with AWS Managed services
DYNATRACE_CLIENTSECRET	Secret string to uniquely identify flowsource as a client for Dynatrace
VERACODE_API_KEY	API_KEY used by flowsource to connect to Veracode
BITBUCKET_CLIENTSECRET	Client Secret value corresponding to a client ID for OAuth2.0 context for secure API communication with BitBucket
APPDYNAMICS_CLIENTSECRET	Client Secret value corresponding to a client ID for OAuth2.0 context for secure API communication with App Dynamics

PLAYWRIGHT_S3_SECRET_KEY	AWS secret key used by Playwright plugin for accessing S3 bucket, where test suite details are stored.
ZEPHYR_ACCESS_TOKEN	Access token required for making api calls to Zephyr plugin.
BITBUCKET_CLOUD_APP_PASSWORD	Password for making api calls to bitbucket server
DATADOG_RUM_CLIENT_TOKEN	Client token for application which is registered with Datadog for RUM testing
SELENIUM_S3_SECRET_KEY	AWS secret key for accessing S3 buckets by Selenium plugin

5. Installation Steps

On running the FlowSource installer using install script, the installation progresses through the following steps

1. It displays the prerequisites.
2. It handles the installer manifests based on the useIngress and useInternalPostgres data set up in values.yaml.
3. It gets the input for AWS ECR or Azure ACR Repo and authenticate the access to ECR or ACR
4. Retrieves the image from flowsource.tar and pushes to the provided Container repository.
5. Confirms that the values are filled in values.yaml.
6. Deploys the flowsource application to the configured Kubernetes cluster using helm.
7. If ingress is used, run kubectl get ingress -n <namespace> to get the endpoint URL. If Loadbalancer is used, run kubectl get svc -n <namepace> to get the loadbalancer endpoint

6. GitHub OAuth Configuration

1. Login to GitHub
2. Create OAuth Application
3. Provide the below details:

   Application Name: <Application Name>

   Homepage URL: http://<DNS NAME>:<PORT NO>

   Authorization callback URL: http://<DNS NAME>:<PORT NO>/api/auth/github/handler/frame

4. Click on Register Application
5. Copy the Client ID generated and update it in app-config.yaml
6. Click on Generate new client secret and update the secret in app-config.yaml

7. Configure GitHub Authentication

1. Login to GitHub and create a OAuth2 Application (Profile->Settings->Developer Settings-OAuth2)
2. Create a new OAuth2 application
3. Enter the below details:

   Application name: <Name of the Application>
   Homepage URL:<URL of the application>
   Authorization callback URL:
   http://<DNS Name>:<Port No.>/api/auth/github/handler/frame

4. Click on Register Application button
5. Copy the Client ID
6. Generate the Client Secret
7. Configure the config-app.yaml with ID and Secret generated on step 5 &6

   clientId: {AUTH_GITHUB_CLIENT_ID}

   clientSecret: {AUTH_GITHUB_CLIENT_SECRET}

8. For service to service auth generate a secret using the below command

   node -p 'require("crypto").randomBytes(24).toString("base64")'

9. Update the BACKEND_SECRET environment variable with the output of the above command.

```
auth:
  keys:
```

- secret: ${BACKEND_SECRET}

Proxied Authentication (using OAuth2 proxy and Github Provider)

1. All the Kubernetes configuration files relevant for this can be found at **proxied-authentication/k8s-yamls**
2. Update the Oauth2 proxy configurations (sensitive-information), by setting the base64 encoded values of the secrets in the fields `OAUTH2_PROXY_CLIENT_ID`, `OAUTH2_PROXY_CLIENT_SECRET` and `OAUTH2_PROXY_COOKIE_SECRET` in the file `oauth-proxy-secrets.yaml`
   - `OAUTH2_PROXY_CLIENT_ID` - Github App client id (in base64)
   - `OAUTH2_PROXY_CLIENT_SECRET` - Github App secret (in base64)
   - `OAUTH2_PROXY_COOKIE_SECRET` - cookie_secret must be 16, 24, or 32 bytes to create an AES cipher (Any random string of mentioned length converted to base64)
3. Updates to DNS:
   - The Oauth2 Proxy configurations in file `oauth-proxy-config.yaml`
     i) `INGRESS_BASE_URL` should be replaced with the actual URL value. Example if the intended URL for FlowSource is https://products.cognizant.com/flowsource, then 'INGRESS_BASE_URL' should be 'https://products.cognizant.com' (Make sure the redirect_url mentioned in the configmap matches the one in the Github OAuth app)
     ii) `INGRESS_DNS` should be replaced with the actual DNS value (Eg.: `products.cognizant.com`)
   - The Ingress configurations in file `flowsource-ingress.yaml` and `oauth-proxy-ingress.yaml` :
     I. `host` (under spec) - Update the value of the host with the appropriate DNS value (By default, the value would be 'localhost')

4. (Code changes) Use the App.tsx provided here separately by overwriting the actual App.tsx present in `packages/app/src/App.tsx`. The changes between the 2 are given below:

   - If using Github provider without Oauth2 proxy:

```
import { SignInPage } from '@backstage/core-components';
import { githubAuthApiRef } from '@backstage/core-plugin-api';
import type { IdentityApi } from '@backstage/core-plugin-api';
import { setTokenCookie } from './cookieAuth';
```

```
    return (
        <SignInPage
          {...props}
          providers={ authProviders }
          title="Select a sign-in method"
          align="center"
          onSignInSuccess={async (identityApi: IdentityApi) => {
            setTokenCookie(
              await discoveryApi.getBaseUrl('cookie'),
              identityApi,
            );
            props.onSignInSuccess(identityApi);
          }}
        />
      );
```

- If using Github provider with Oauth2 proxy:

```
import { ProxiedSignInPage } from '@backstage/core-components';

      return (
        <ProxiedSignInPage {...props} provider="oauth2Proxy" />
      );
```

5. (Optional) If you want to change the namespace of the oauth2-proxy related k8s resources, then,
   - update the namespace in all the files (oauth-proxy-*.yaml).
   - Also, update the line `nginx.ingress.kubernetes.io/auth-url: http://oauth-proxy.oauth2-proxy.svc.cluster.local:4180/oauth2/auth` with (proper namespace replaced) `nginx.ingress.kubernetes.io/auth-url: http://oauth-proxy.NAMESPACE.svc.cluster.local:4180/oauth2/auth` in file `flowsource-ingress.yaml`.
6. If your current namespace (in your current context) is not the namespace of flowsource, then, set the flowsource's namespace in file `flowsource-ingress.yaml` (in field `metadata.namespace`)
7. Run the installer script with the updated App.tsx file.

8. Apply all the k8s yamls at **proxied-authentication/k8s-yamls**
9. Test the application using the Ingress URL

8. Azure AD Authentication provider

Azure Portal App Registration

1. Login to Azure Portal > App registrations, and find your existing app registration, or create a new one.

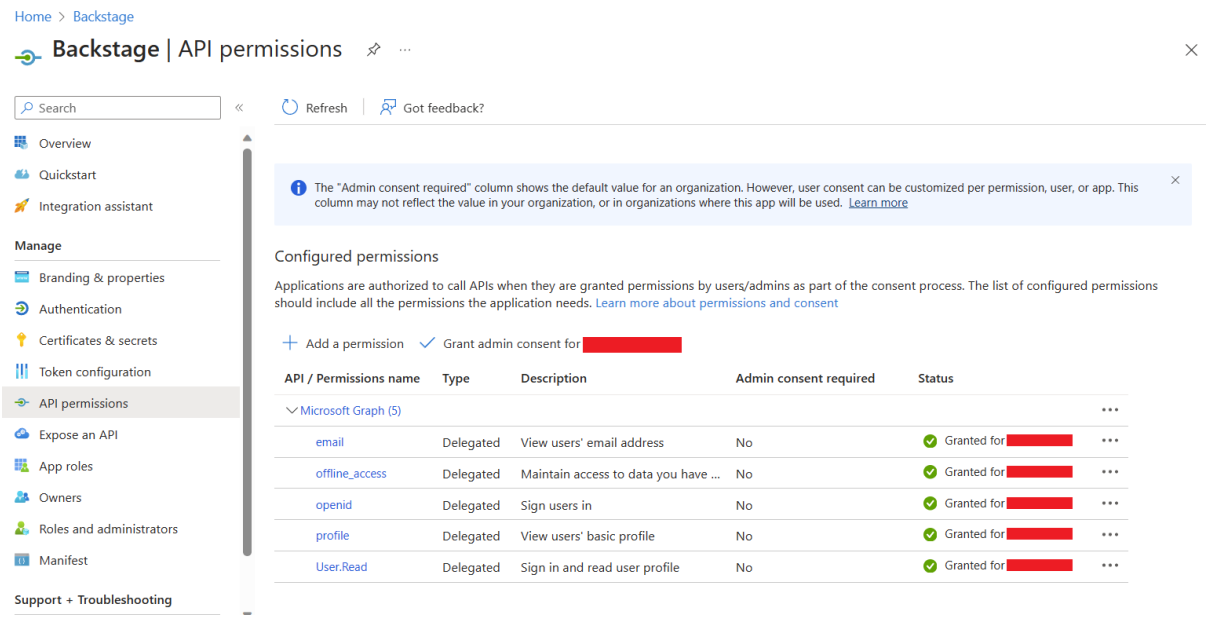
   On your app registration's overview page, add a new Web platform configuration, with the configuration:

   Redirect URI: https://<Flowsource DNS>/api/auth/microsoft/handler/frame

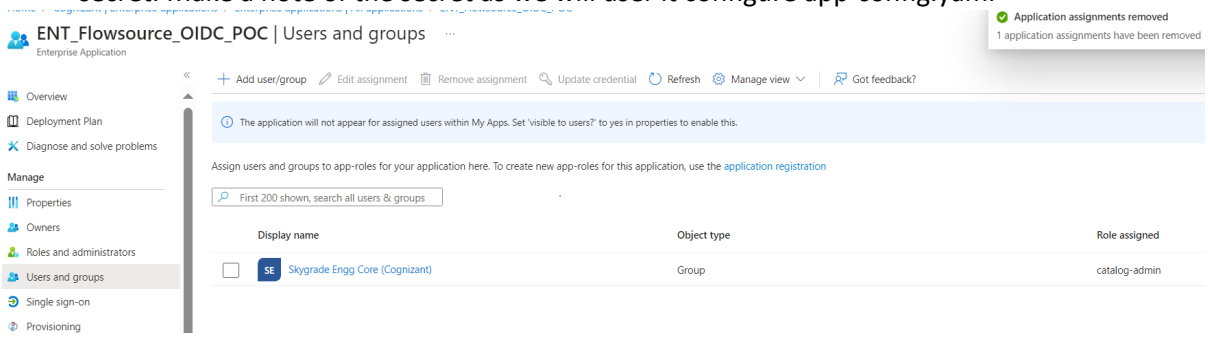
   Front-channel logout Url: blank

   Implicit grant and hybrid flows: All unchecked

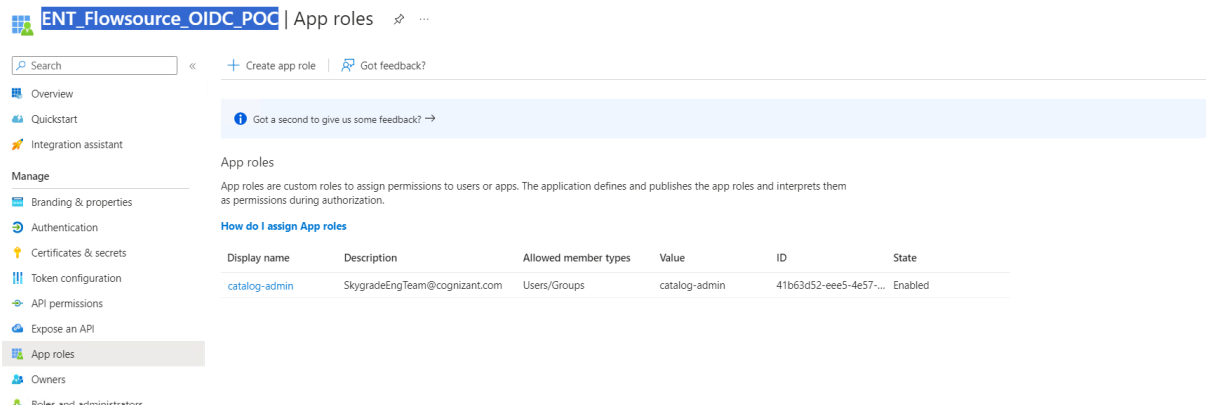
2. On the API permissions tab, click on Add Permission, then add the following Delegated permission for the Microsoft Graph API.
   - email
   - offline_access
   - openid
   - profile
   - User.Read
3. To grant admin consent, a directory admin will need to come to this page and click on the Grant admin consent for COMPANY NAME button.



4. Go to the **Certificates & Secrets** page, then the **Client secrets** tab and create a new client secret. Make a note of the secret as we will user it configure app-config.yaml



5. Assign catalog-admin role to the required team which requires admin access to the Flowsource. In this example we have used Skygrade Engg Core team



Flowsource Configuration

9. app-config.yaml

```
auth:
  environment: development
  providers:
    microsoft:
      development:
        clientId: ${AZURE_CLIENT_ID}
        clientSecret: ${AZURE_CLIENT_SECRET}
        tenantId: ${AZURE_TENANT_ID}
        domainHint: ${AZURE_TENANT_ID}
```

Note: Make sure that the Flowsource URL is configured with HTTPS enabled for the call back from Microsoft Azure to work.

10. Insert entry for the role added in step 5, in the Flowsource Postgres backstage_plugin_auth database

If the database is running on docker/podman then follow the below steps or use pgAdmin to execute the insert statement.

I. Get the postgres container id
   docker | podman ps

II. Login to the container
   docker | podman exec -it <container-id> bash

III. Connect to the postgres database
   psql -h localhost -p 5432 -U postgres -d backstage_plugin_auth

IV. Execute the insert statement to enable catalog-admin role for the user belonging to skygrade-enginnering-core team

   INSERT INTO public.role_mappings(flowsource_role, auth_provider, auth_provider_role)
   VALUES ('catalog-admin', 'microsoft', catalog-admin');

Code Changes:

1. Add the Azure AD on the login page

   Package -> app -> src -> App.tsx

```
const microsoftAuthProvider: SignInProviderConfig = {
  id: 'microsoft-auth-provider',
  title: 'Microsoft Azure AD',
  message: 'Sign in using Microsoft Azure AD',
  apiRef: microsoftAuthApiRef,
};

const githubAuthProvider: SignInProviderConfig = {
  id: 'github-auth-provider',
  title: 'GitHub',
  message: 'Sign in using GitHub',
  apiRef: githubAuthApiRef,
};

type AuthProvider = "guest" | SignInProviderConfig;

const authProviders: AuthProvider[] = [
  microsoftAuthProvider,
  githubAuthProvider,
]

const app = createApp({
  apis,
  components: {
    SignInPage: props => {
      const discoveryApi = useApi(discoveryApiRef);
      return (
        <SignInPage
          {...props}
          providers={ authProviders }
          title="Select a sign-in method"
          align="center"
          onSignInSuccess={async (identityApi: IdentityApi) => {
            setTokenCookie(
              await discoveryApi.getBaseUrl('cookie'),
              identityApi,
            );
            props.onSignInSuccess(identityApi);
          }}
        />
      );
    },
  },
```

2. Package-core -> backend -> src -> plugins -> auth.ts

```
      microsoft: providers.microsoft.create({
        signIn: {
          resolver: async (info, ctx) => {
            //Decode the ID Token
            const idToken: any = decodeJwt(info.result.params.id_token!);
            if (!info.profile.email) {
              throw new Error(
                'Login failed, user profile does not contain an email',
              );
            }
            const userEntityRef = stringifyEntityRef({
              kind: 'User',
              name: info.profile.email,
              namespace: DEFAULT_NAMESPACE,
            });
            let roleArray: any = [];
            if(idToken.roles != null && idToken.roles != undefined){
              roleArray = idToken.roles;
            } else {
              console.log("No role found");
            }
            const userRefs = await getUpdatedUserRefs(roleArray, 'microsoft',
roleMappingDatabaseService); // Fetch the updated user references
            return ctx.issueToken({
              claims: {
                sub: userEntityRef,
                ent: userRefs,
              },
            });
          },
        },
      }),
```

11. Updates on app-config.yaml

```
auth:
  # see https://backstage.io/docs/auth/ to learn about auth providers
  environment: development
  providers:
    github:
      development:
        clientId: ${AUTH_GITHUB_CLIENT_ID}
        clientSecret: ${AUTH_GITHUB_CLIENT_SECRET}
        githubOrganization: ${GITHUB_ORGANIZATION}
    microsoft:
      development:
```

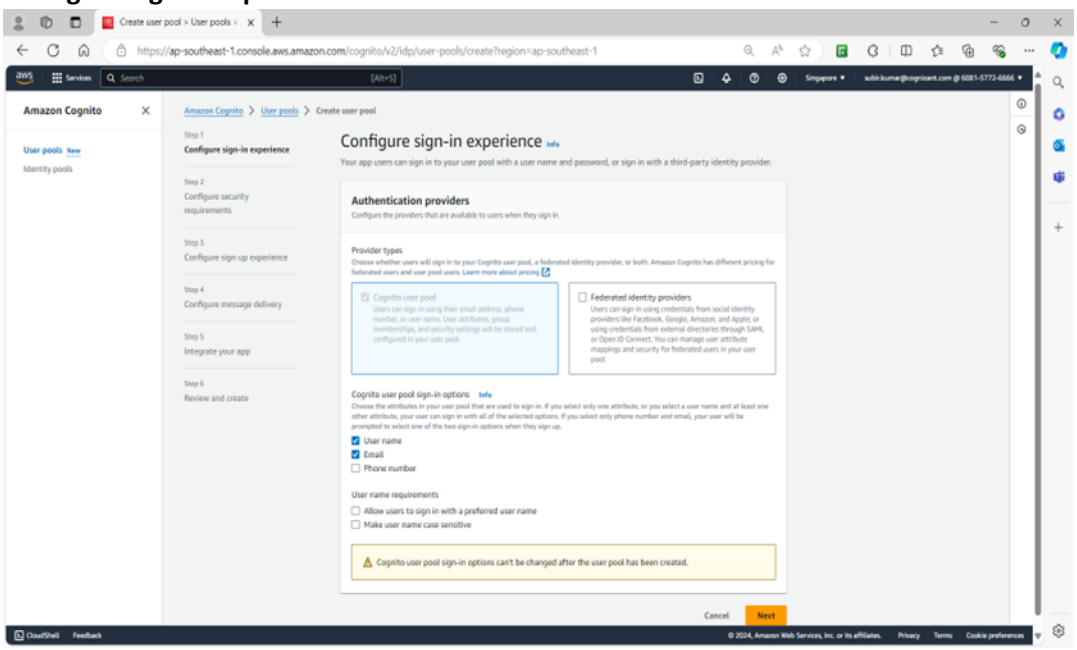
```
        clientId: ${AZURE_CLIENT_ID}
        clientSecret: ${AZURE_CLIENT_SECRET}
        tenantId: ${AZURE_TENANT_ID}
        domainHint: ${AZURE_TENANT_ID}
```

9. AWS Cognito Authentication provider

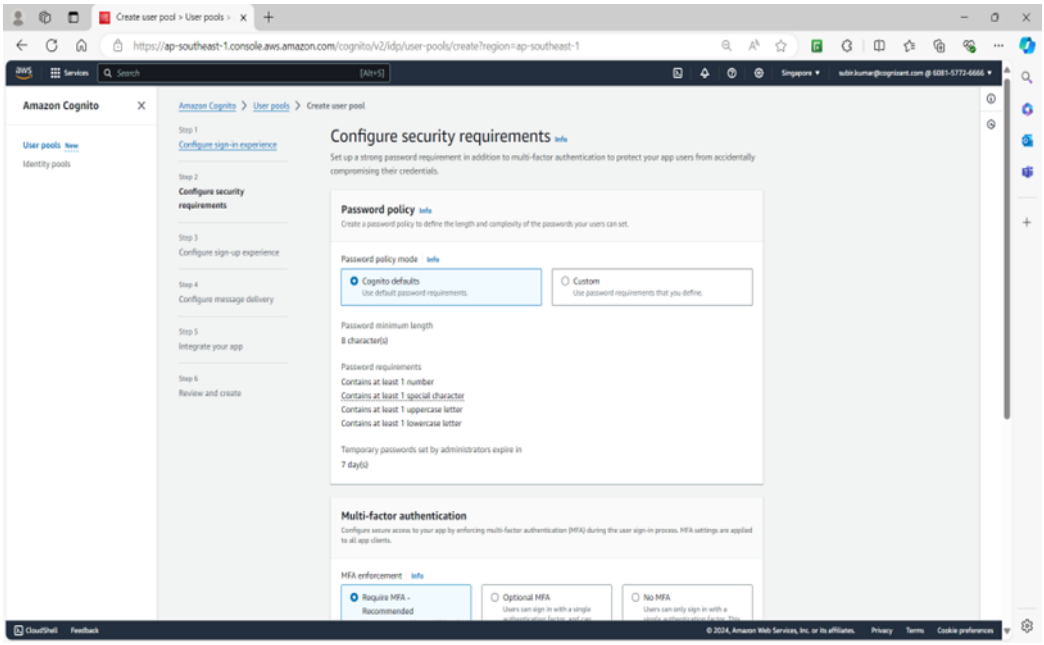
AWS Cognito User Pool Configuration

The below steps are for reference for configuration of AWS Cognito in AWS. This can be modified based on project and security requirements.

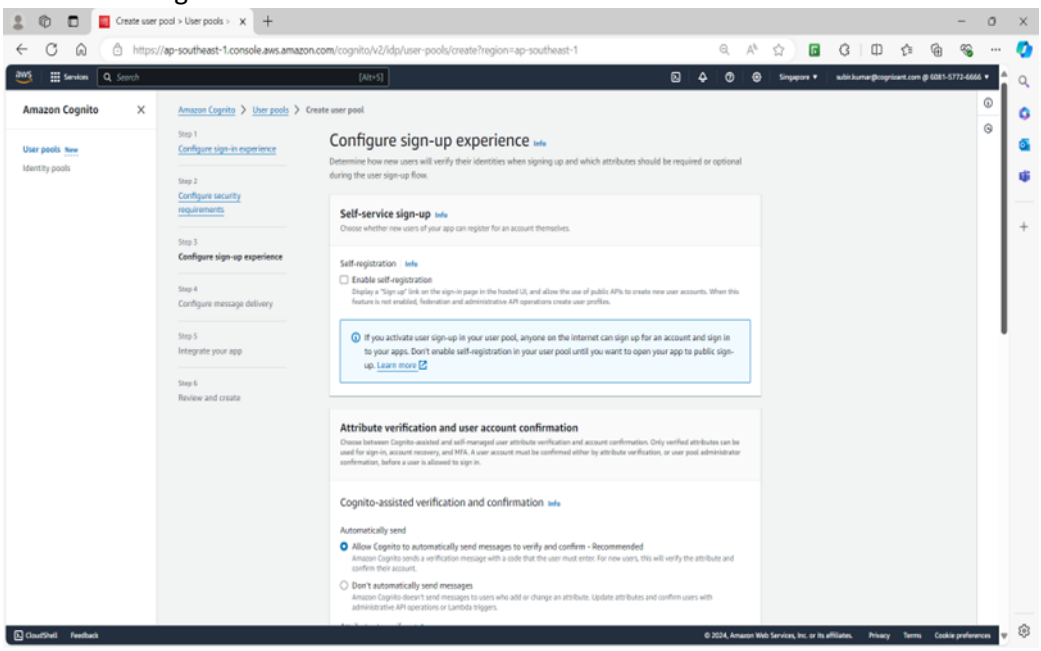
1. Login to AWS Portal > Cognito, and find your existing Cognito user pool, or create a new one.
   a) **Configure Sign in Experience:** check User name and Email



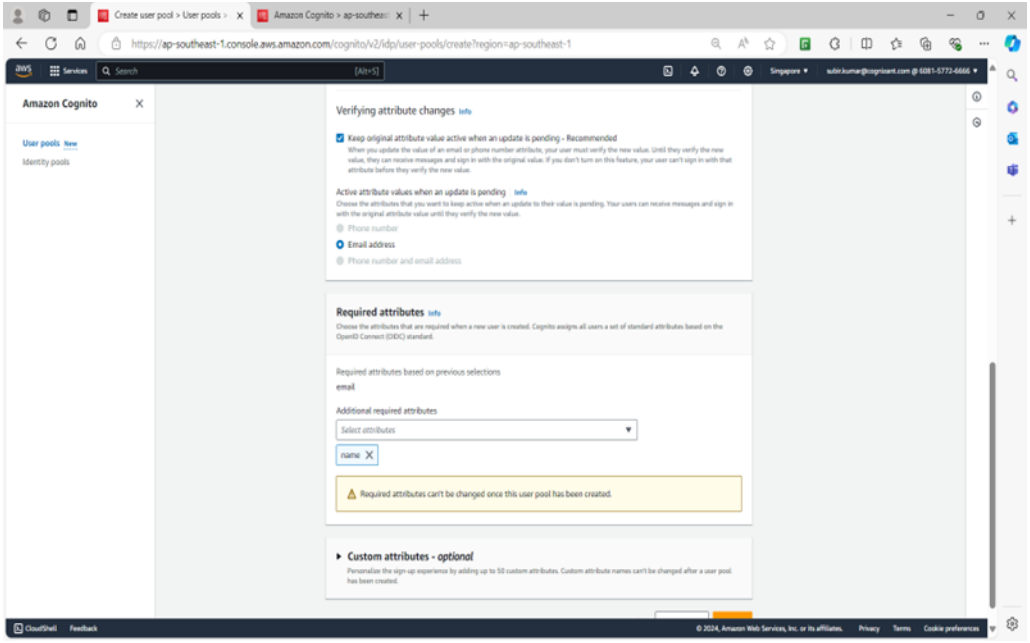
   b) **Configure Security Requirements:** Select Cognito default password, custom password and configure MFA based on security requirements.



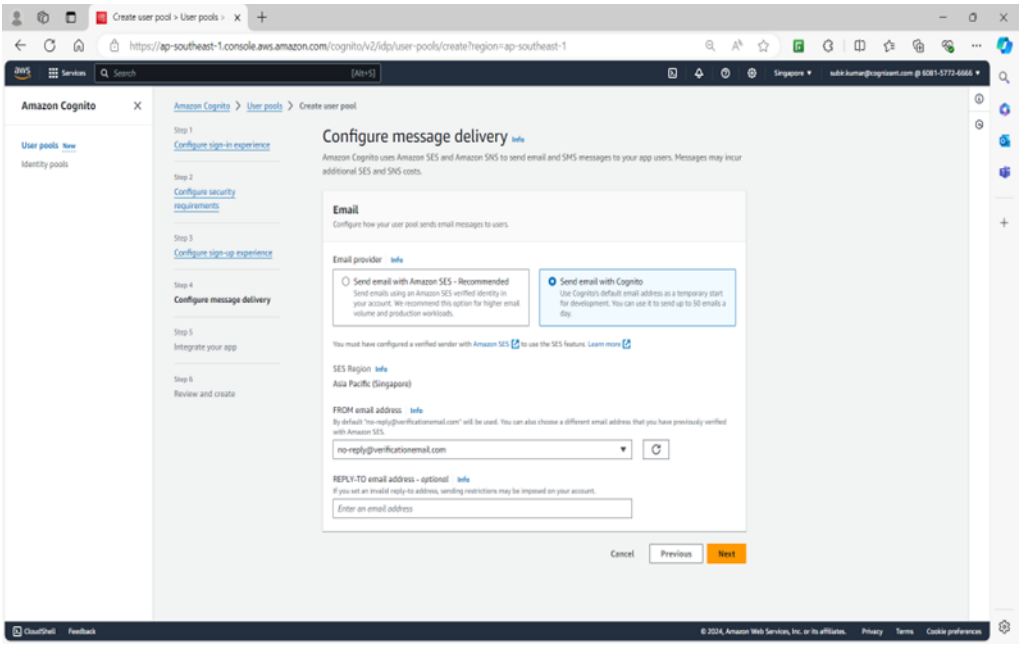
c) Disable self-registration



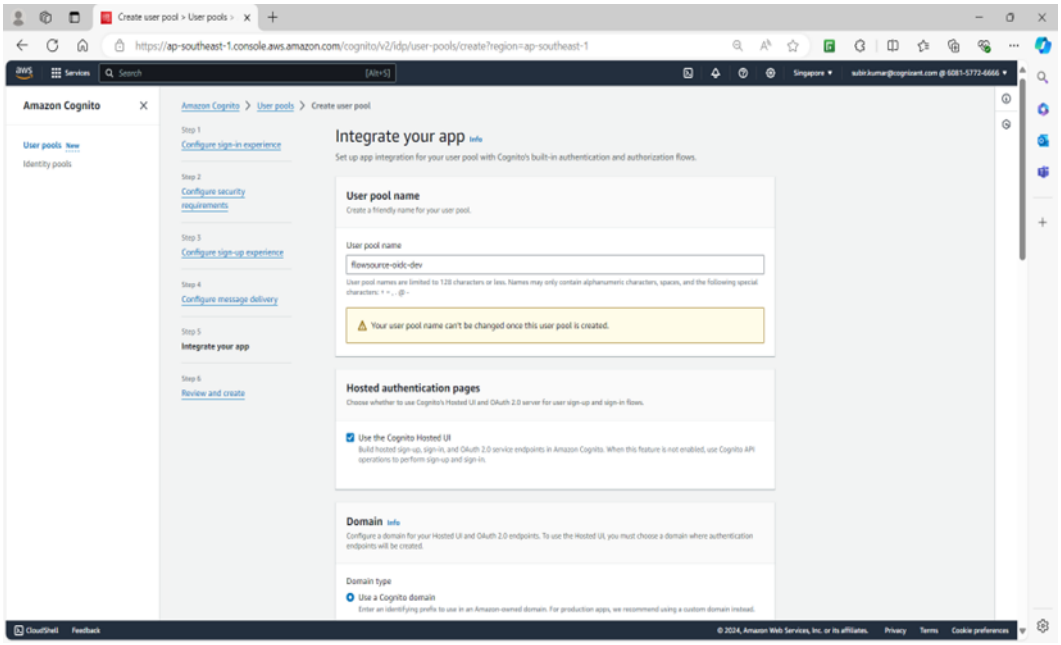
d) Add required attribute – name or any additional attribute required for the user



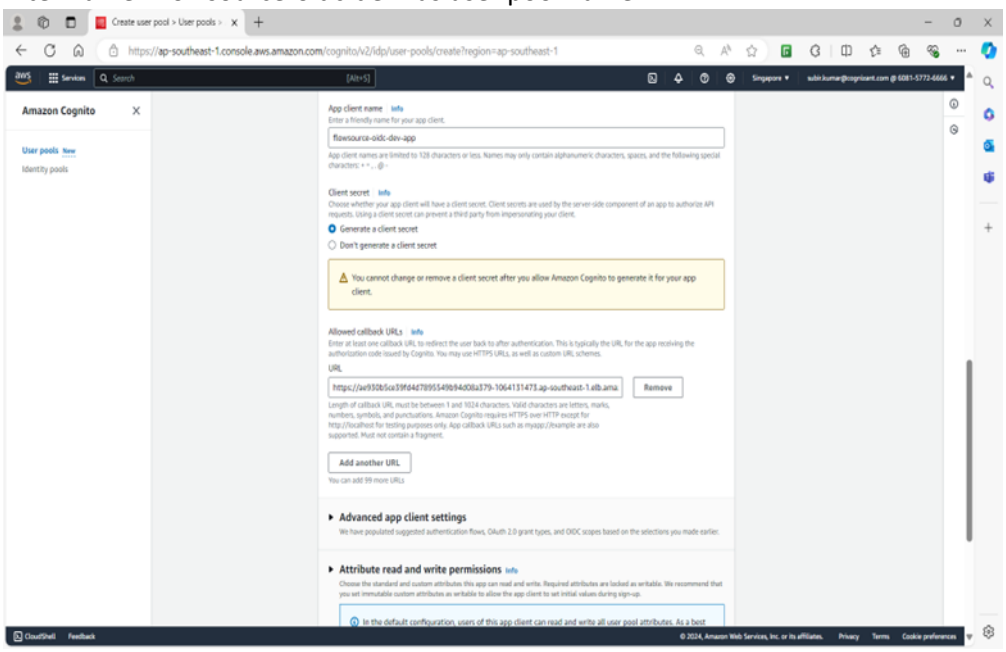
e) Configure Message Delivery SES or Cognito

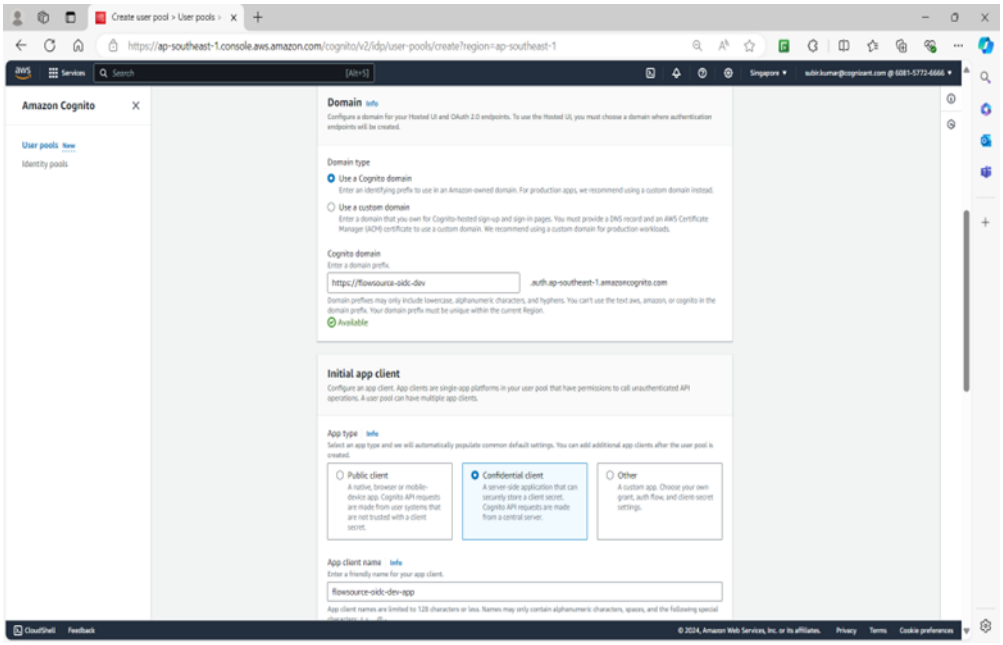


f) App Integration



g) Enter name “flowsource-oidc-dev” as user pool name

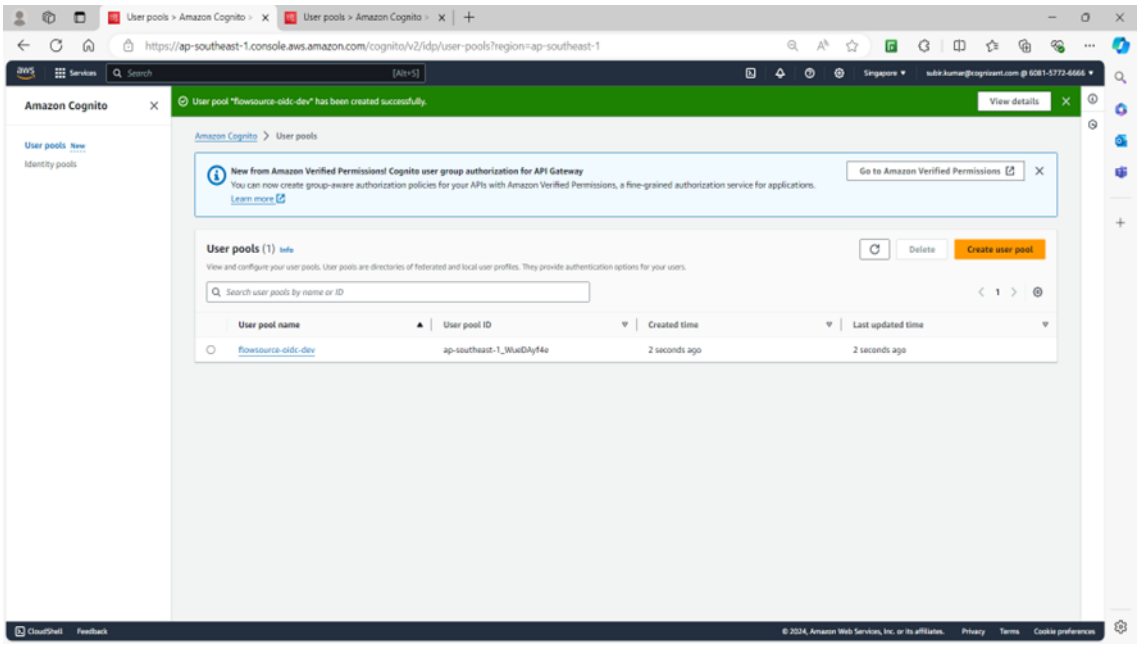


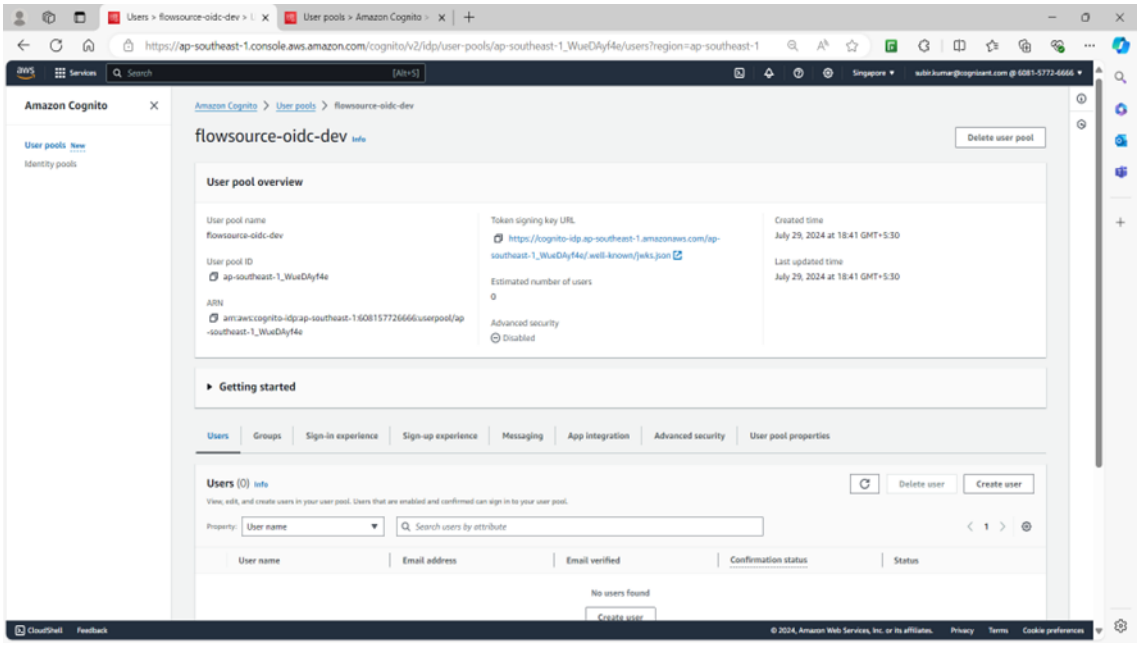


Update the call back URL -

Redirect URI: https://<Flowsource DNS>:<Flowsource Port_No>/api/auth/microsoft/handler/frame

Note: Make sure that the Flowsource URL is configured with HTTPS enabled for the call back from AWS Cognito to work.





h) Copy the Token Signing URL
i) Click on the app name under App Integration
j) Copy the Client ID and Client Secret
k) Edit the "**Hosted UI**" Add phone, email, open id, profile under OpenID Connect Scopes and save
l) Create a user "catalog-admin" & "catalog-user"
m) Create a group "catalog-admin" & "catalog-user"
n) Map the user to the corresponding groups. Add catalog-admin user to catalog-admin group and add catalog-user to catalog-user group.

Flowsource Configuration

1. app-config.yaml

```
auth:
  environment: development
  session:
    secret: ${AUTH_SESSION_SECRET}
  providers:
    cognito:
      development:
        metadataUrl: ${AUTH_AWS_COGNITO_METADATA_URL}
        tokenUrl: ${AUTH_AWS_COGNITO_TOKEN_URL}
        authorizationUrl: ${AUTH_AWS_COGNITO_AUTHORIZATION_URL}
        clientId: ${AUTH_AWS_COGNITO_CLIENT_ID}
```

clientSecret: ${AUTH_AWS_COGNITO_CLIENT_SECRET}

prompt: auto

The values can be found or generated as mentioned above:

AUTH_SESSION_SECRET

The auth.session.secret can be generated by executing the below command:

node -p 'require("crypto").randomBytes(24).toString("base64")'

AUTH_AWS_COGNITO_METADATA_URL

In AWS Cognito service-> click on the AWS Cognito user pool under app integration. Copy the Token Signing Key URL and modify the URL to get the Open ID configuration URL. For example:

https://cognito-idp.ap-southeast-1.amazonaws.com/ap-southeast-1_hKHdvIBK8/.well-known/openid-configuration

AUTH_AWS_COGNITO_AUTHORIZATION_URL

Open the Open ID configuration URL from the previous step in a browser and copy the authorization_endpoint URL and update it in the configuration. For example:

https://flowsource-oidc-dev-app.auth.ap-southeast-1.amazoncognito.com/oauth2/authorize

AUTH_AWS_COGNITO_TOKEN_URL

Open the Open ID configuration URL from the previous step in a browser and copy the token_endpoint URL and update it in the configuration. For example:

https://flowsource-oidc-dev-app.auth.ap-southeast-1.amazoncognito.com/oauth2/token

AUTH_AWS_COGNITO_CLIENT_ID

In AWS Cognito service-> click on the AWS Cognito user pool under app integration. Copy the Client ID

AUTH_AWS_COGNITO_CLIENT_SECRET

In AWS Cognito service-> click on the AWS Cognito user pool under app integration. Copy the Client Secret

2. Insert an entry for the group added in AWS Cognito in the Flowsource Postgres backstage_plugin_auth database

   If the database is running on docker/podman then follow the below steps or use pgAdmin to execute the insert statement.

I) Get the postgres container id
docker | podman ps

II) Login to the container
docker | podman exec -it <container-id> bash

III) Connect to the postgres database
psql -h localhost -p 5432 -U postgres -d backstage_plugin_auth

IV) Execute the insert statement to enable catalog-admin role for the user belonging to catalog-admin group in AWS Cognito.

INSERT INTO public.role_mappings(flowsource_role, auth_provider, auth_provider_role) VALUES ('catalog-admin', cognito, catalog-admin');

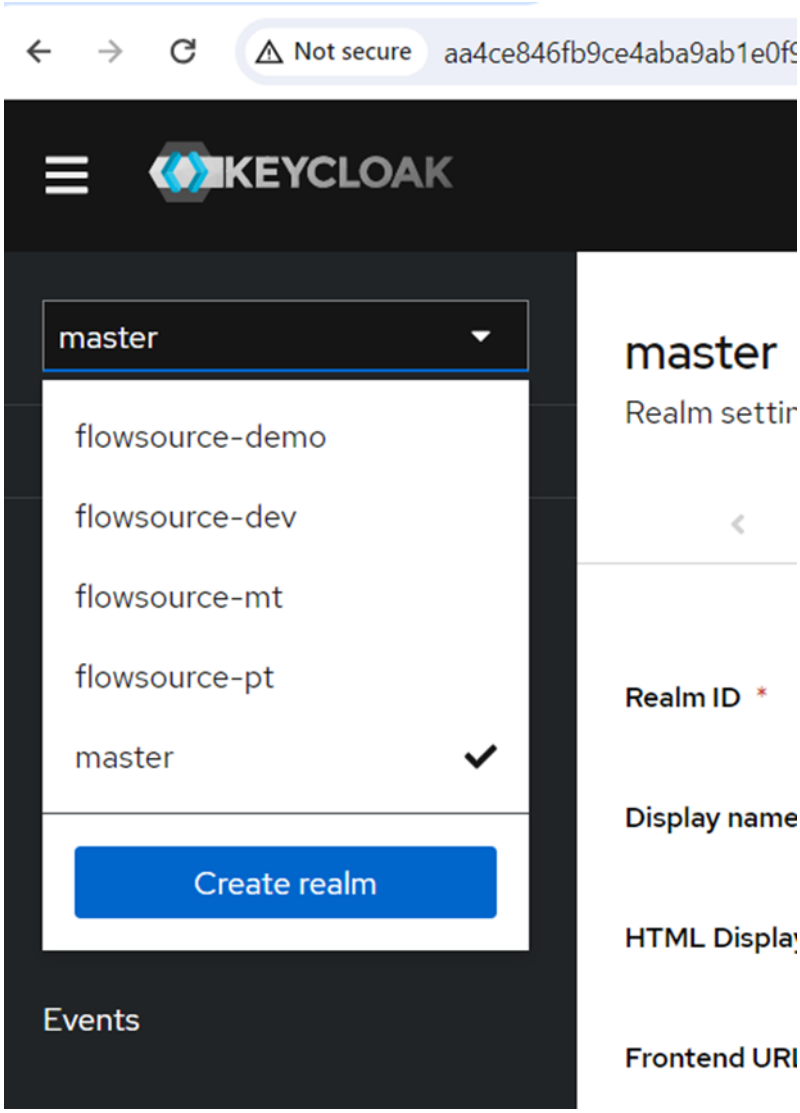
V) Execute the insert statement to enable catalog-user role for the user belonging to catalog-user group in AWS Cognito.

VI) INSERT INTO public.role_mappings(flowsource_role, auth_provider, auth_provider_role) VALUES ('catalog-user', cognito, catalog-user');

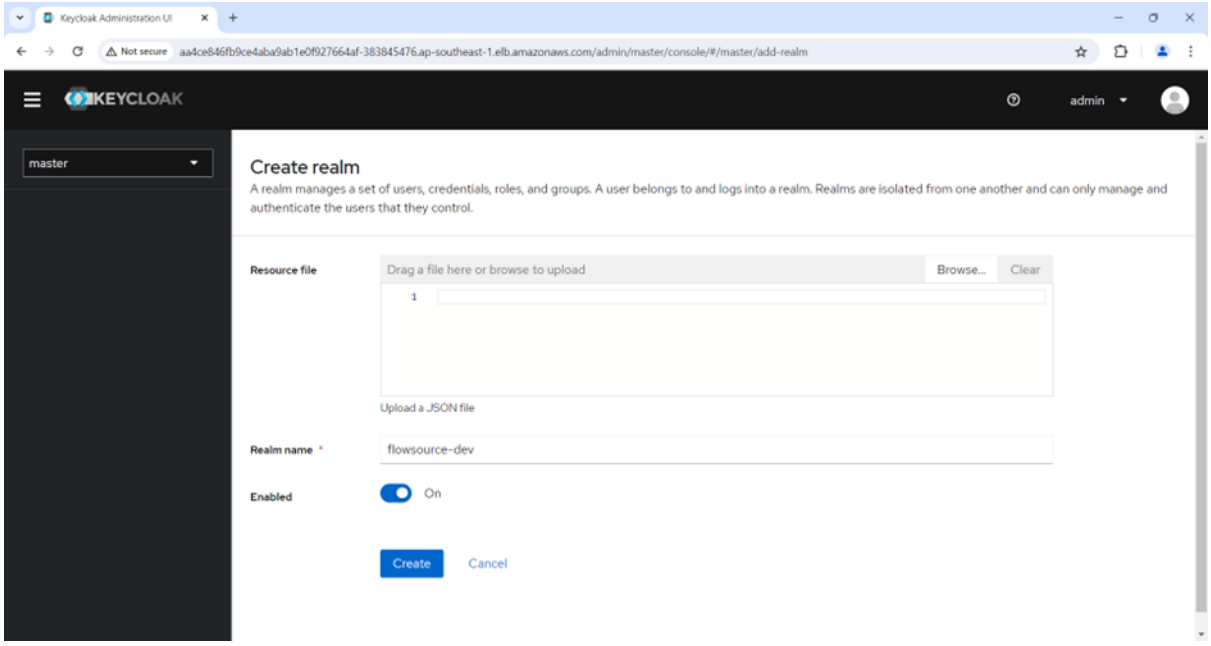
10. Keycloak Authentication provider

Keycloak OIDC Client Configuration

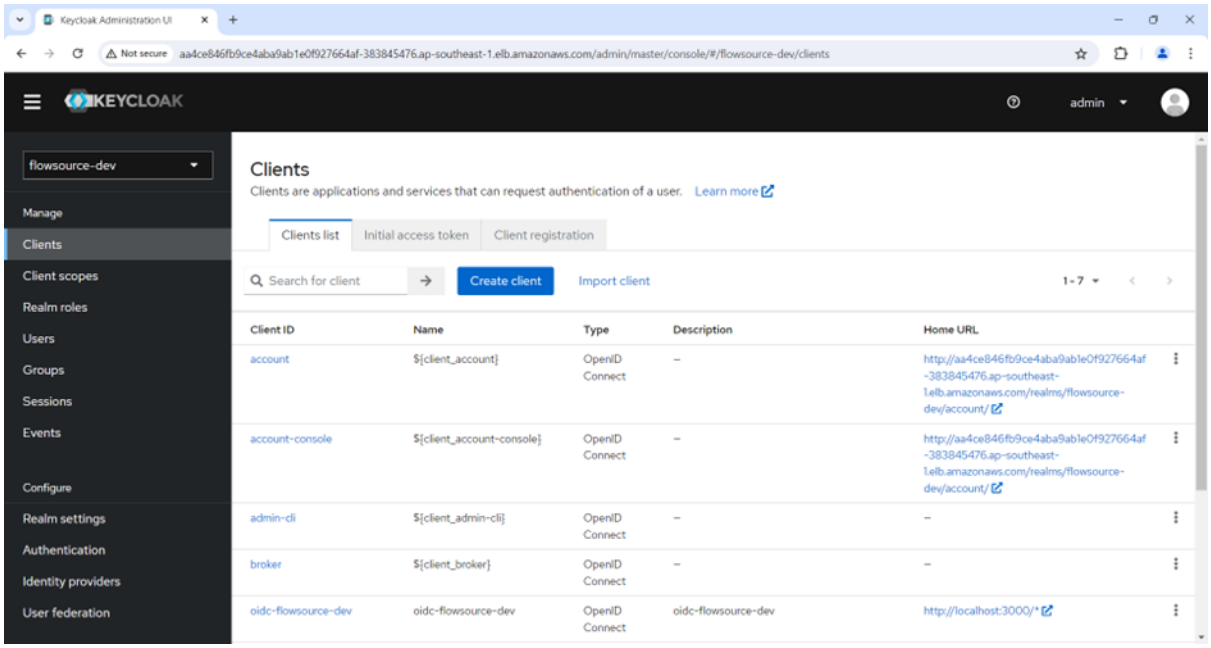
1. Login to the Keycloak admin console
2. From the left navigation Select the dropdown which is populated as master by default. Click on create realm



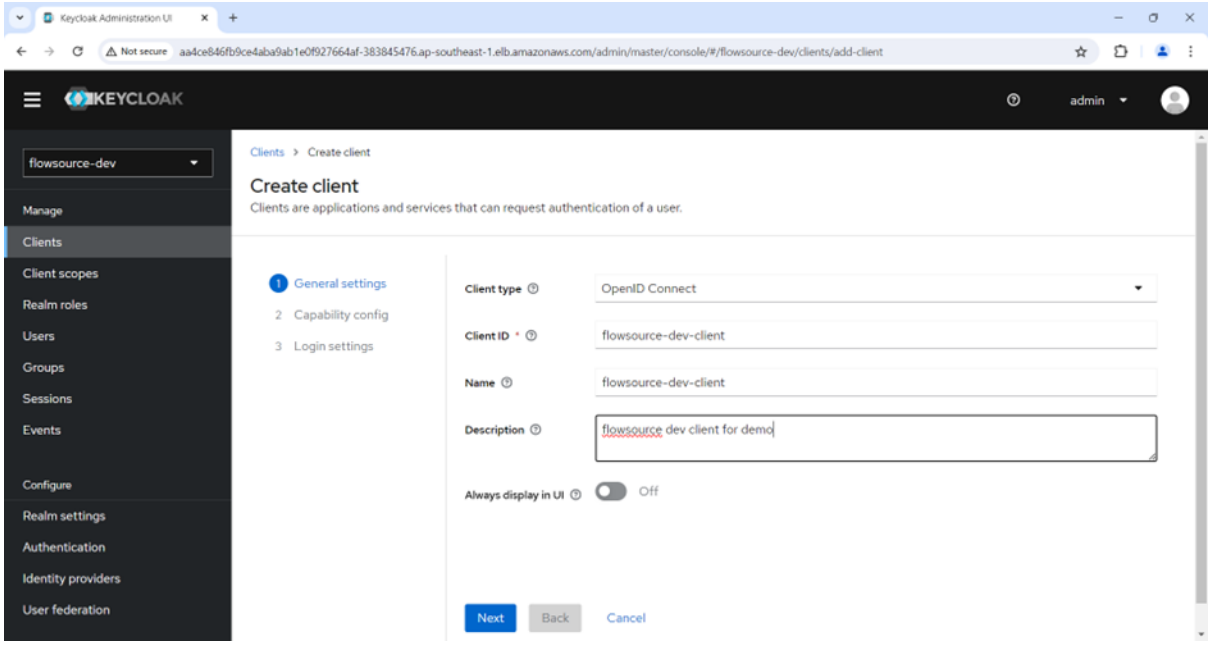
3. Enter the custom realm name. In this case it has been entered it as ‘flowsource-dev’



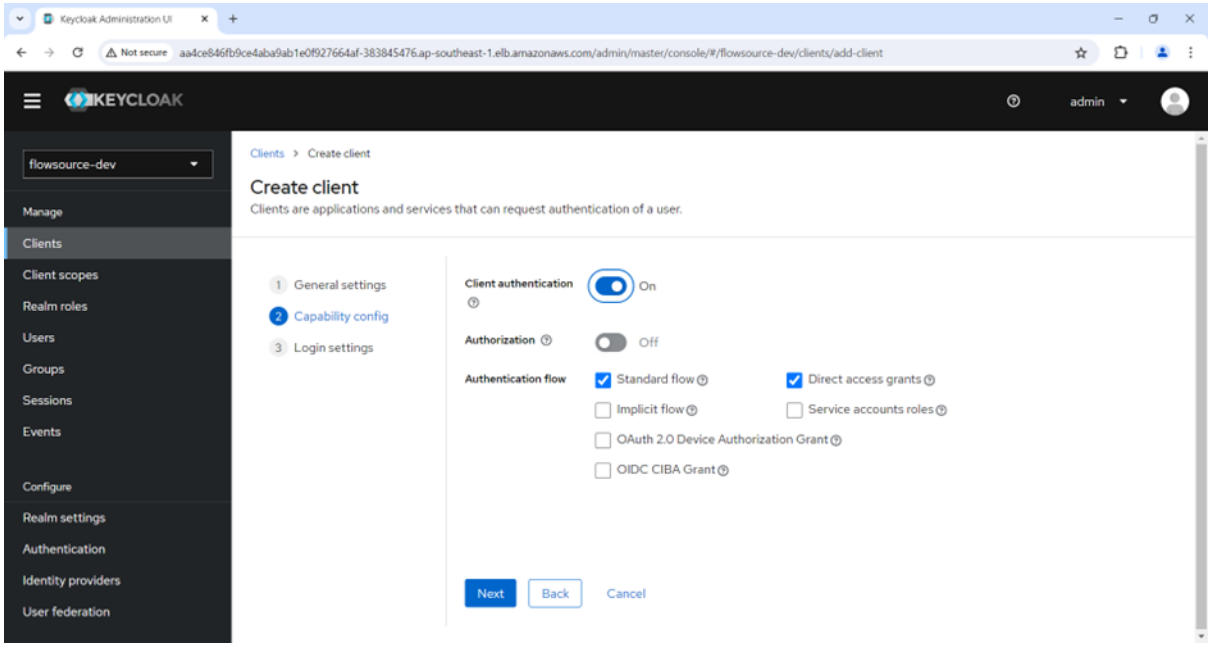
4. On the top left corner, select the newly create realm name from the drop down
5. Click on clients from the left navigation
6. Click on create client



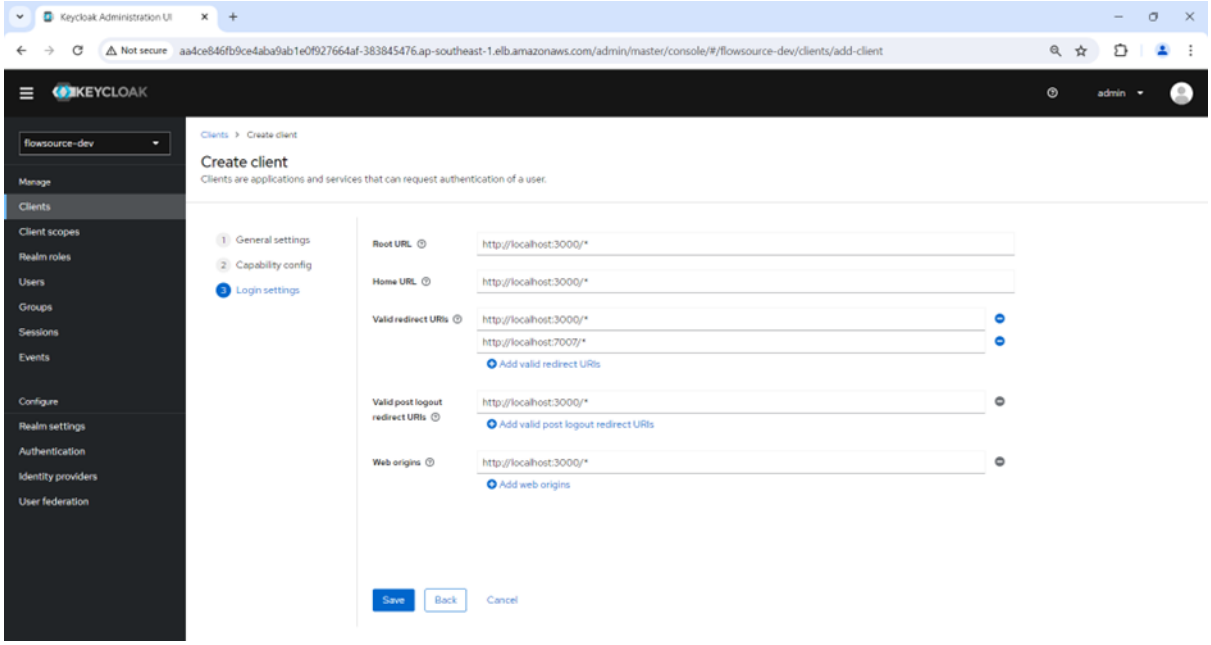
7. Select Client type as OIDC Connect and enter the Client ID & Client Name as mentioned below:



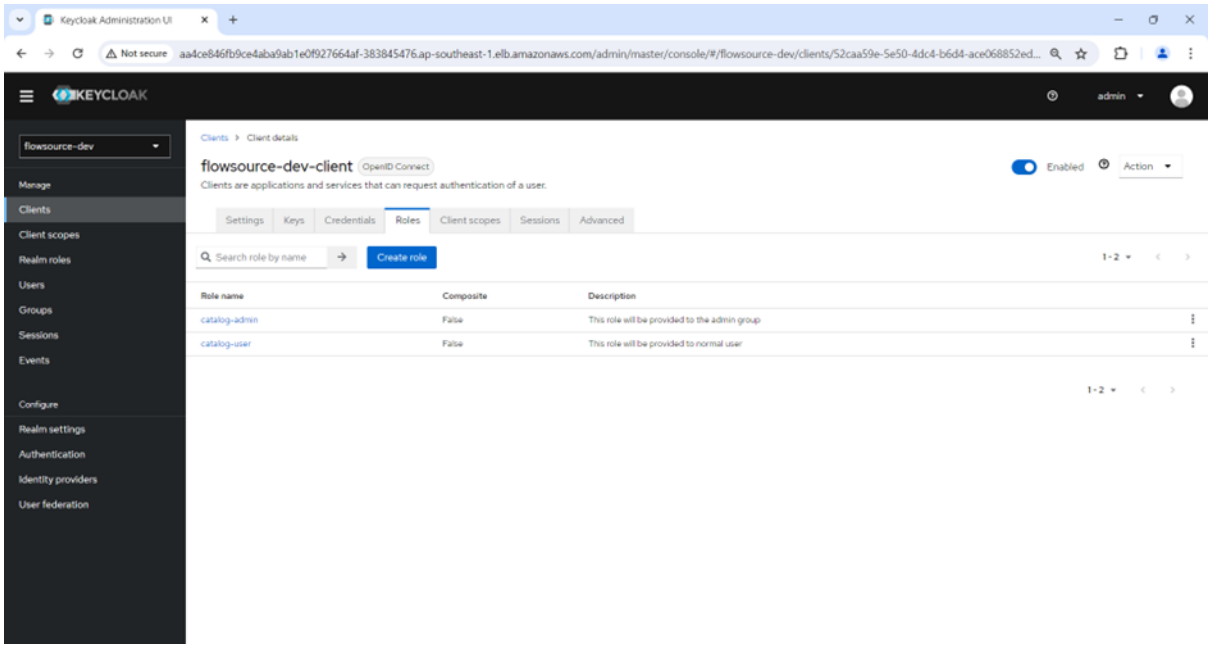
8. Click on next and enable client authentication



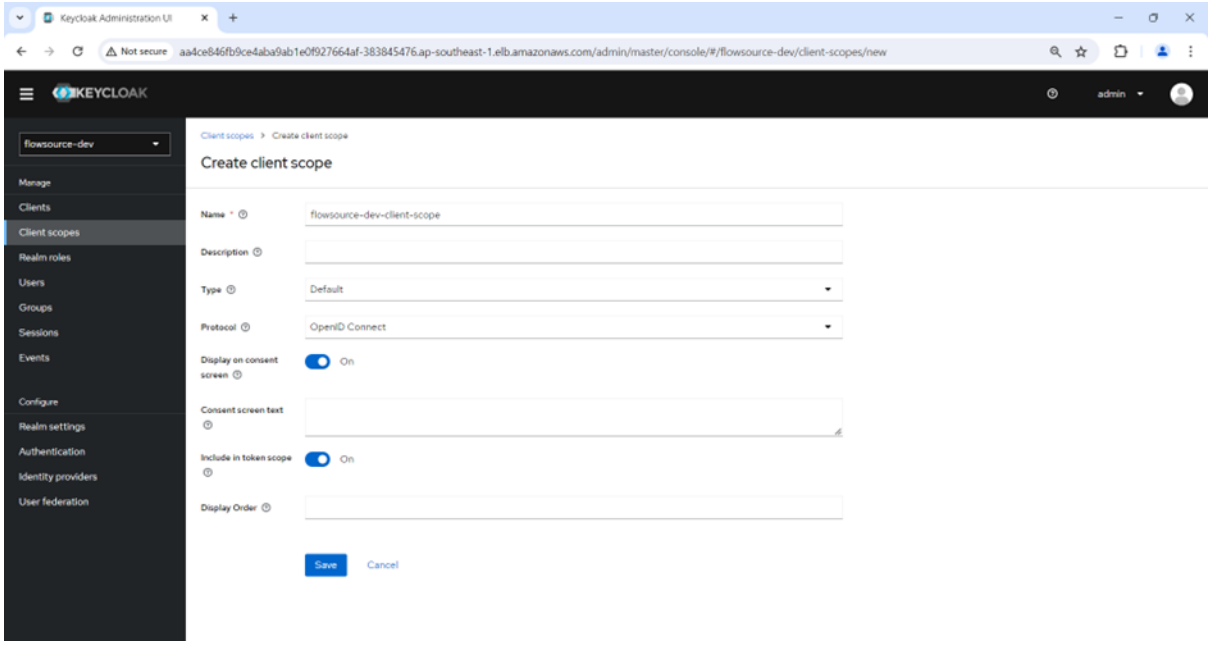
9. Click on next and enter the root URL, redirect URL, Logout URL and Web Origin URL. Please refer to the screenshot for details:



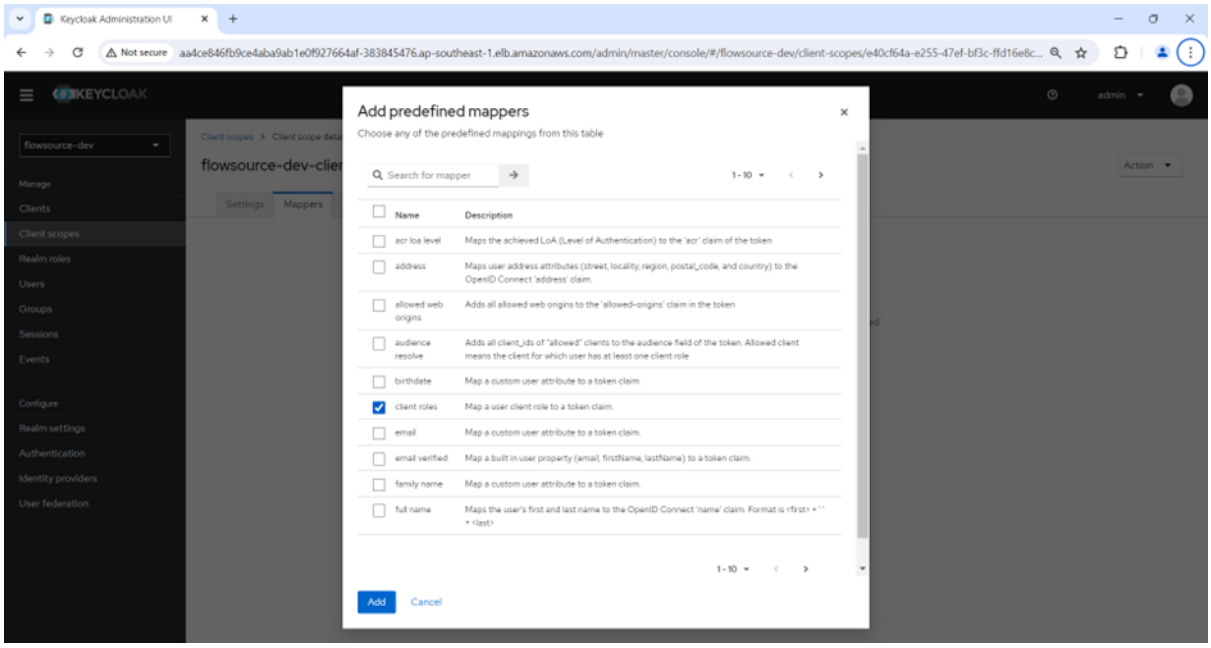
10. Click on Save
11. Click on Left Navigation->Clients->Click on ‘flowsource-dev-client’->Click on Roles tab and create the below roles:



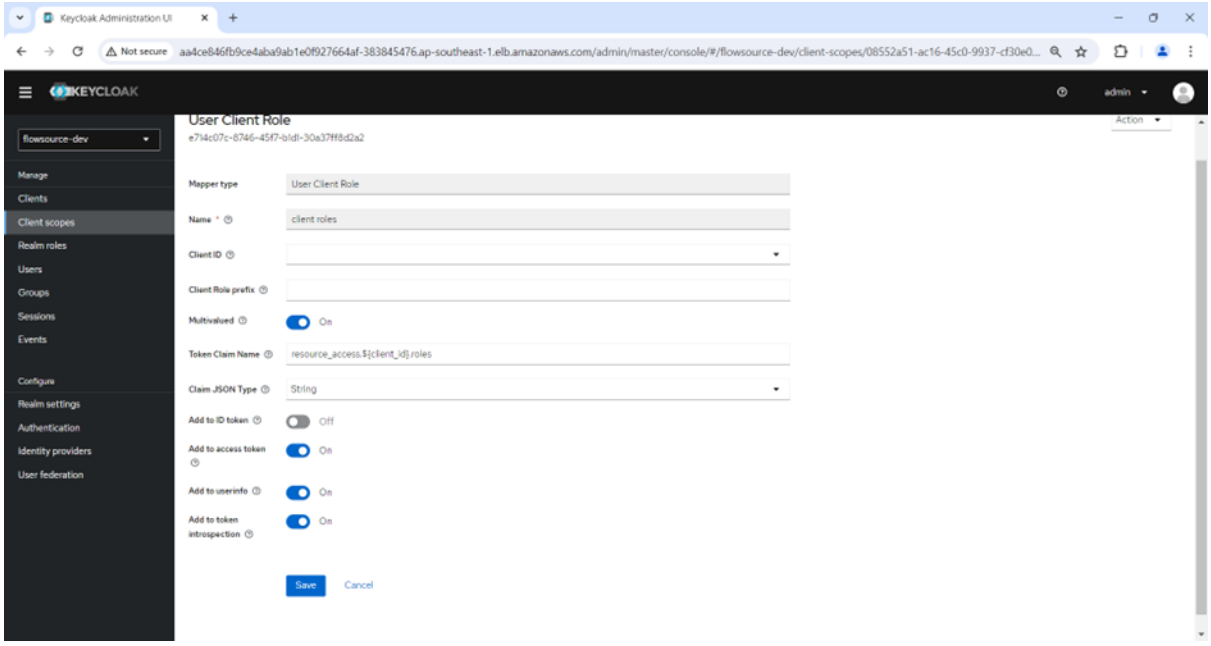
12. On the left Navigation ->Click on Client Scopes-> Enter name as ‘flowsource-dev-client-scope’-> Select type as default



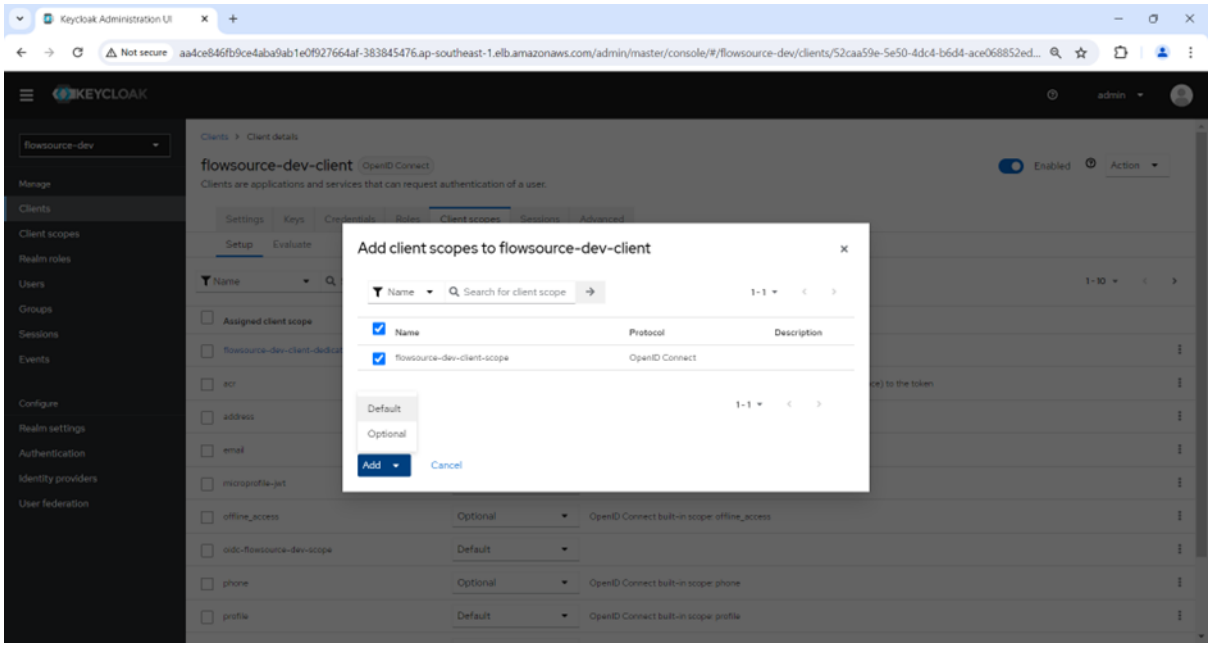
13. Click on Save
14. Click on the Mapper tab -> Click on the predefined roles-> Select Client roles from the list and click on add.



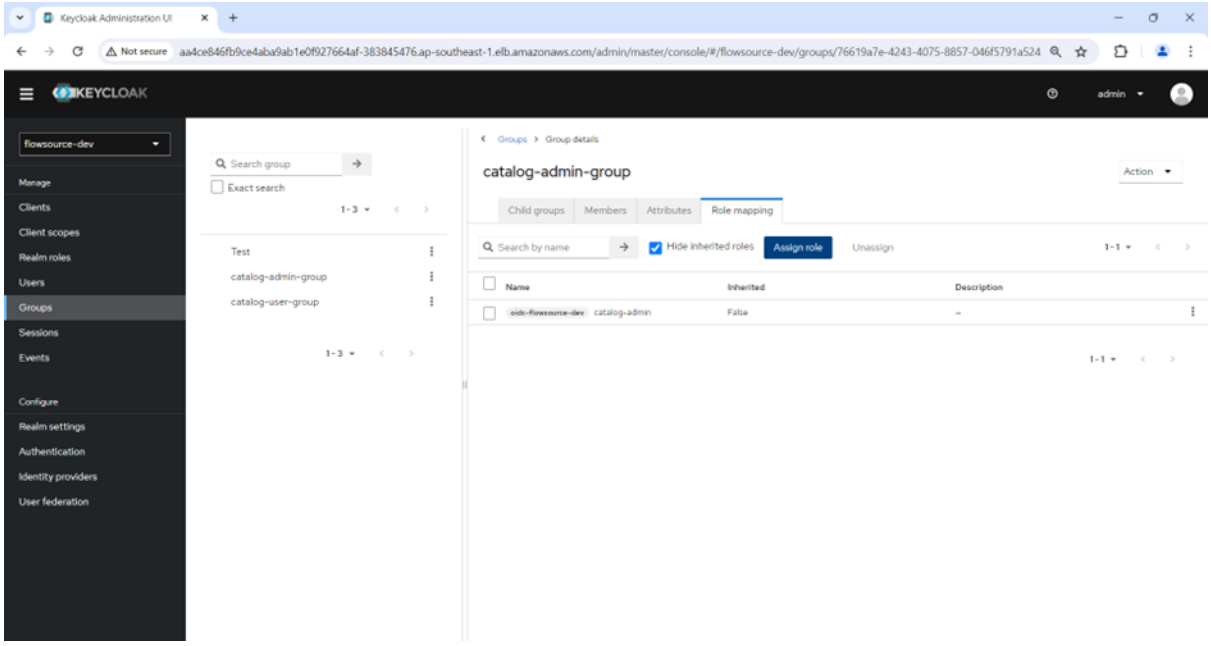
15. Click on the client roles under the mapper. Enable add the userinfo and click on save



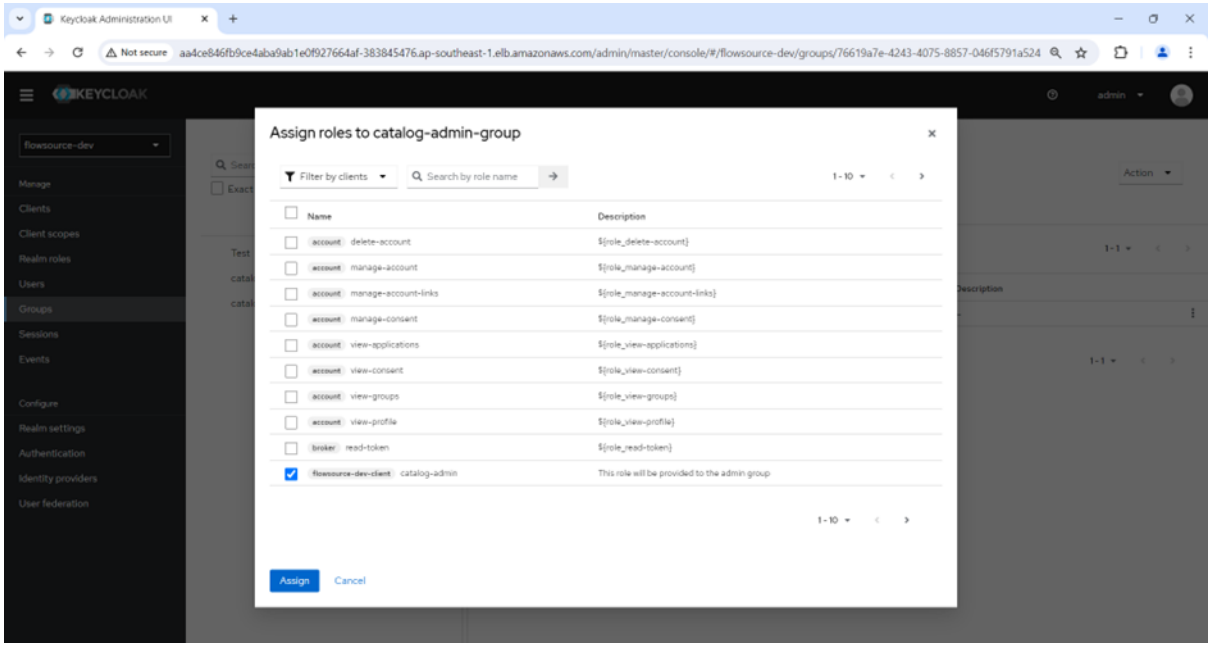
16. On the left navigation ->Click on client-> Click on ‘flowsource-dev-client’->Click on the Client Scopes tab->Click on add client scopes->Select the ‘flowsource-dev-client-scope’->Click on Add as default



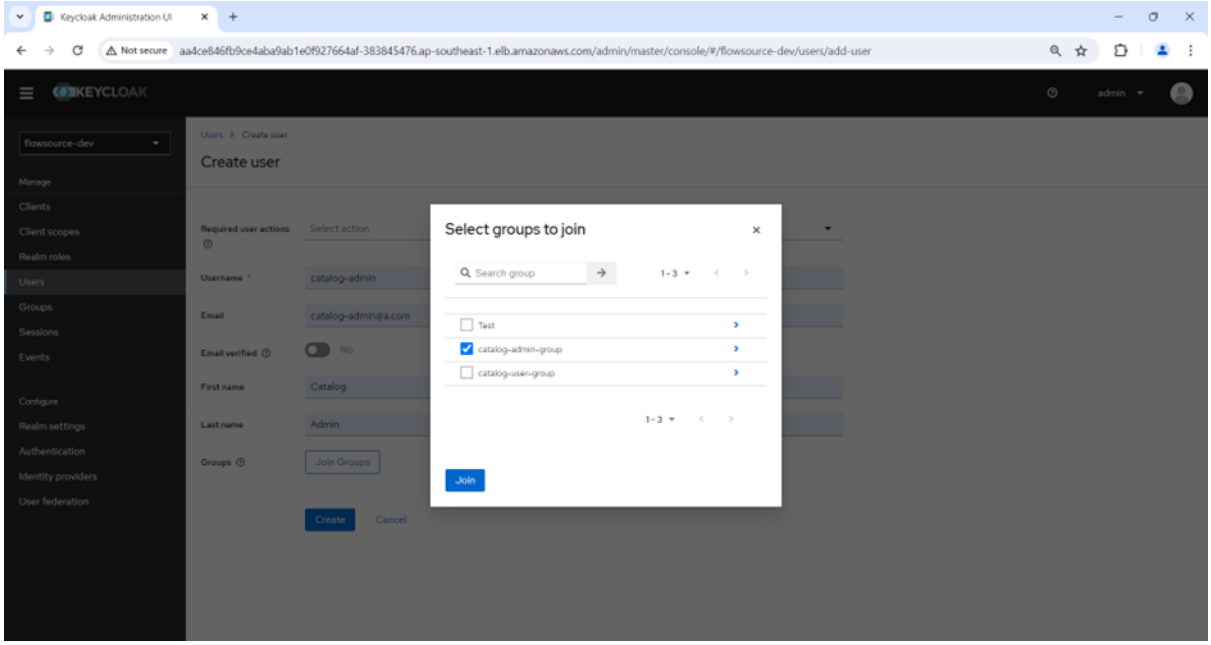
17. Create a group ‘catalog-admin-group’ and click on the Role Mapping tab



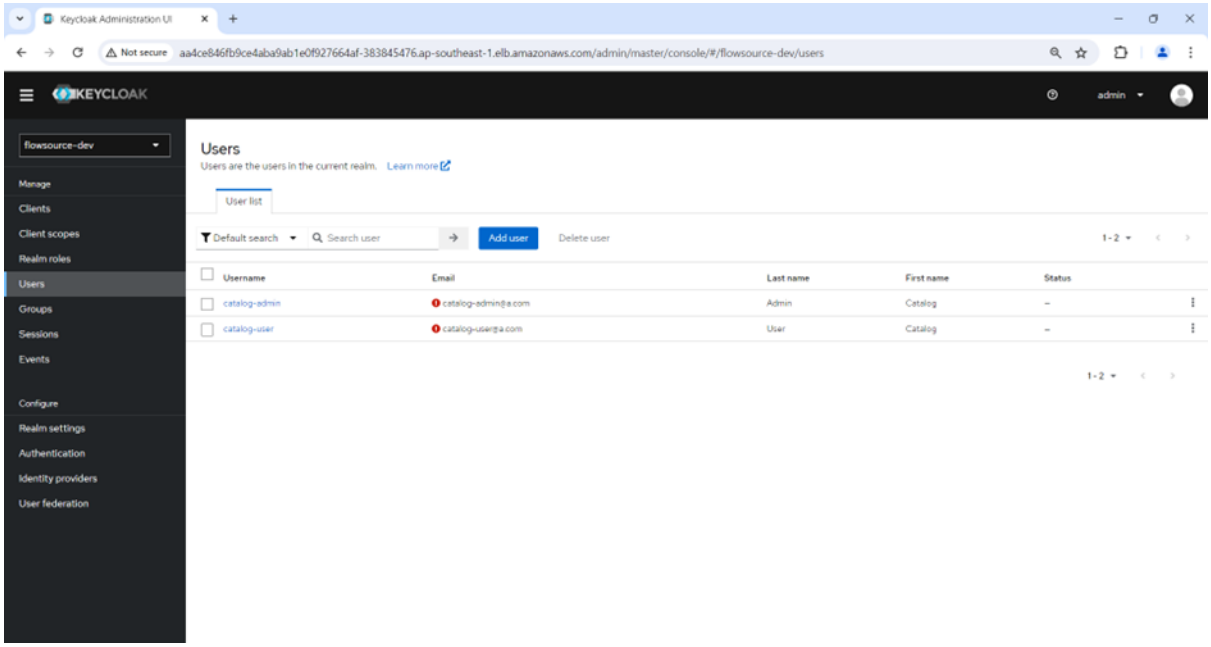
18. Click on assign role->Select Filter by client->Select the checkbox with flowsource-dev-client catalog-admin-> Click on Assign



19. Follow the step 17 & 18 to create a group name – catalog-user-group and ->Select the checkbox with flowsource-dev-client catalog-user-> Click on Assign
20. Create two users – catalog-admin and catalog-user. On the left Navigation click on Users ->Add Users.



21. Enter the user details and click on join group -> and select catalog-admin-group to assign admin role to the user.
22. Similarly create a catalog-user and -> and select catalog-user-group to assign admin role to the user.



23. Set passwords of the user by clicking on the username and click on Credentials tab and set password

Flowsource Configuration

1. app-config.yaml

```
auth:

  environment: development
```

```
session:
  secret: ${AUTH_SESSION_SECRET}
providers:
  keycloak:
    development:
      metadataUrl: ${KEYCLOAK_METADATA_URL}
      tokenUrl: ${KEYCLOAK_TOKEN_URL}
      authorizationUrl: ${KEYCLOAK_AUTHORIZATION_URL}
      clientId: ${KEYCLOAK_CLIENT_ID}
      clientSecret: ${KEYCLOAK_CLIENT_SECRET}
      prompt: auto
```

The values can be found or generated as mentioned above:

AUTH_SESSION_SECRET

The auth.session.secret can be generated by executing the below command:

node -p 'require("crypto").randomBytes(24).toString("base64")'

KEYCLOAK_METADATA_URL

In Keycloak admin console-> Select 'flowsource-dev' realm-> In left navigation click on Realm Setting -> Click on Open ID Endpoint Configuration-> Click on the hyperlink and copy the URL and update the metadata URL in the Flowsource configuration. For example:

http://aa4ce846fb9ce4aba9ab1e0f927664af-383845476.ap-southeast-1.elb.amazonaws.com/realms/flowsource-dev/.well-known/openid-configuration

KEYCLOAK_AUTHORIZATION_URL

Open the Open ID Endpoint configuration URL from the previous step in a browser and copy the authorization_endpoint URL and update it in the configuration. For example:

http://aa4ce846fb9ce4aba9ab1e0f927664af-383845476.ap-southeast-1.elb.amazonaws.com/realms/flowsource-dev/protocol/openid-connect/auth

KEYCLOAK_TOKEN_URL

Open the Open ID configuration URL from the previous step in a browser and copy the token_endpoint URL and update it in the configuration. For example:

http://aa4ce846fb9ce4aba9ab1e0f927664af-383845476.ap-southeast-1.elb.amazonaws.com/realms/flowsource-dev/protocol/openid-connect/token

KEYCLOAK_CLIENT_ID

The client ID is entered during the client creation. The same must be used in this parameter. For example: 'flowsource-pt-client' is the client id in this case.

KEYCLOAK_CLIENT_SECRET

In Keycloak Admin Console-> In left navigation click on Clients-> Click on 'flowsource-pt-client'-> Under the credential tab copy the Client Secret.

2. Insert entry for the group added in AWS Cognito in the Flowsource Postgres backstage_plugin_auth database

   If the database is running on docker/podman then follow the below steps or use pgAdmin to execute the insert statement.

I. Get the postgres container id
   docker | podman ps

II. Login to the container
   docker | podman exec -it <container-id> bash

III. Connect to the postgres database
   psql -h localhost -p 5432 -U postgres -d backstage_plugin_auth

IV. Execute the insert statement to enable catalog-admin role/group for the user belonging to catalog-admin group in Keycloak.

   INSERT INTO public.role_mappings(flowsource_role, auth_provider, auth_provider_role) VALUES ('catalog-admin', keycloak, catalog-admin');

V. Execute the insert statement to enable catalog-user role/group for the user belonging to catalog-user group in Keycloak.

   INSERT INTO public.role_mappings(flowsource_role, auth_provider, auth_provider_role) VALUES ('catalog-user', keycloak, catalog-user');

Note: If the Keycloak Authentication provider is not used then we can safely comment the Keycloak configuration as highlighted below in yellow.

Filename: app-config.yaml

```
auth:
  environment: development
  session:
    secret: ${AUTH_SESSION_SECRET}
  providers:
    #keycloak:
```

```
#development:
  #metadataUrl: ${KEYCLOAK_METADATA_URL}
  #tokenUrl: ${KEYCLOAK_TOKEN_URL}
  #authorizationUrl: ${KEYCLOAK_AUTHORIZATION_URL}
  #clientId: ${KEYCLOAK_CLIENT_ID}
  #clientSecret: ${KEYCLOAK_CLIENT_SECRET}
  #prompt: auto
```

11. Flowsource and Authentication provider role/group mapping

Role-based access to the components is enabled in Flowsource. The users who are part of a team/Group and the same Group is set as the owner of catalog-info.yaml will only be able to view the components and the templates in the Flowsource dashboard.

For this configuration let's consider a Group – catalog-user and the catalog-user is the owner specified in the catalog-info.yaml file

1. Update the catalog-info.yaml with the owner as 'Group: catalog-user' or 'Group:default/catalog-user'.
   For example –
   owner: Group:default/catalog-user

   Note: If there are multiple owner attributes then update all of them

2. Map the teams/Group in the database by making a database entry mentioned below:

   a) connect to the backstage_plugin_auth database by running the below command on the pod/container:
      psql -h localhost -p 5432 -U <db-username> -d backstage_plugin_auth

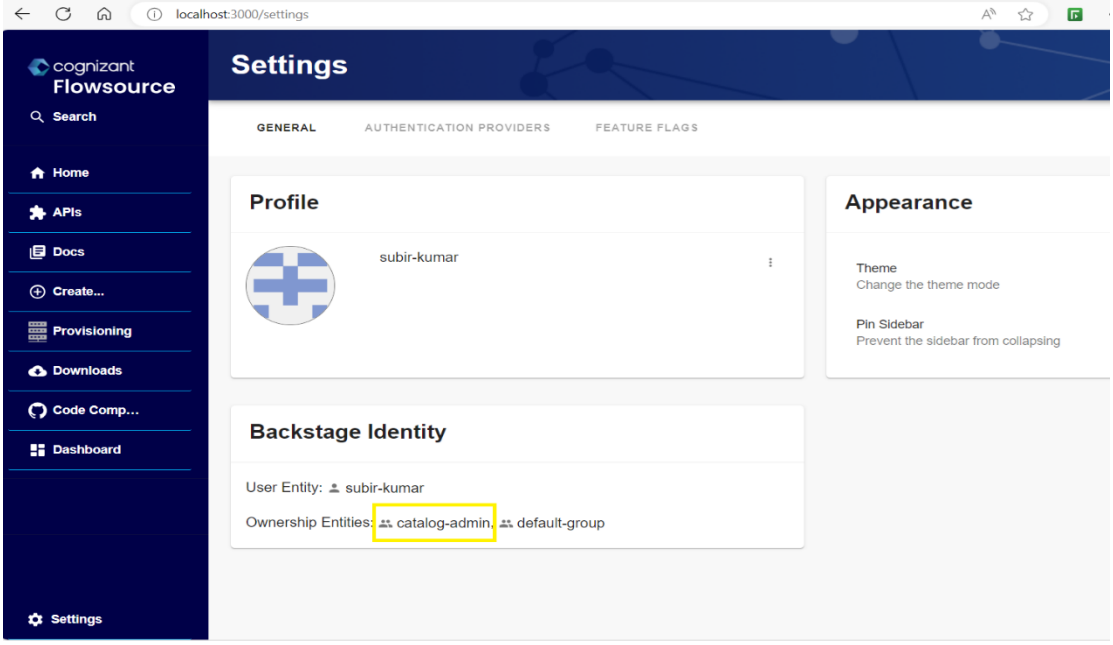
   b) Configure catalog-admin
   INSERT INTO public.role_mappings(flowsource_role, auth_provider, auth_provider_role) VALUES ('catalog-admin', <auth provider>, <team/role name>);

   Example:
   INSERT INTO public.role_mappings(flowsource_role, auth_provider, auth_provider_role) VALUES ('catalog-user', 'github', 'skygrade-team');

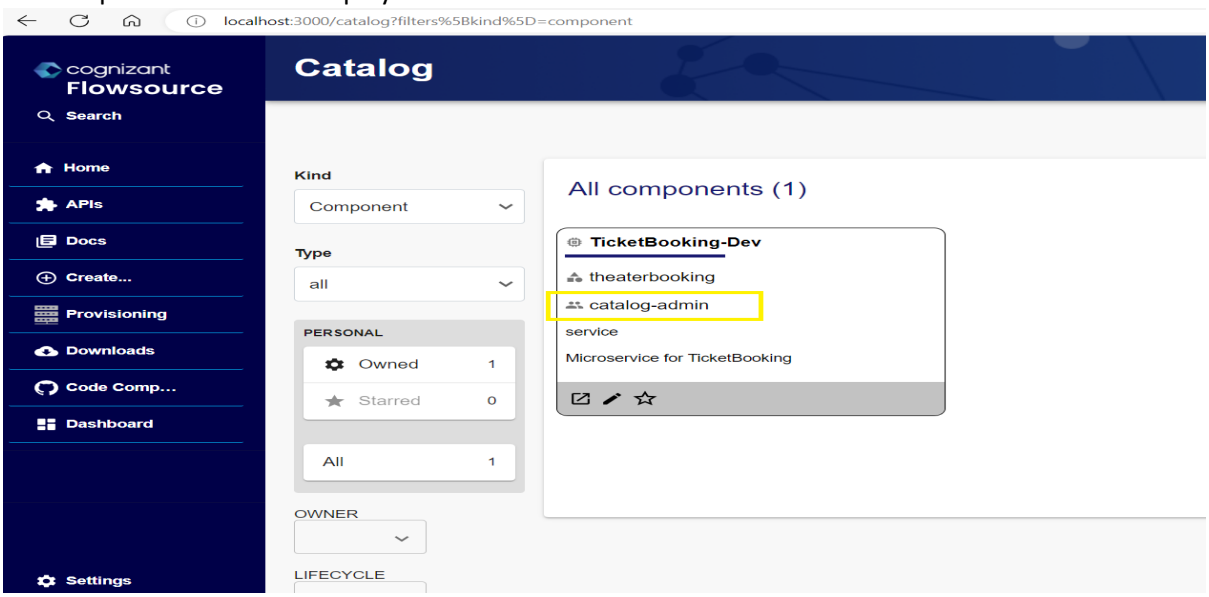
   c) Configure catalog-user
   INSERT INTO public.role_mappings(flowsource_role, auth_provider, auth_provider_role) VALUES ('catalog-user', <auth provider>, <team/role name>);

Example:
INSERT INTO public.role_mappings(flowsource_role, auth_provider, auth_provider_role) VALUES ('catalog-user', 'github', 'skygrade-support');

3. Once the mapping is done the Group will reflect on the user's setting page.



4. The catalog-info.yaml information will be displayed on the dashboard. If the user does have the Group specified in the owner. Those catalog-info.yaml components and templates will not be displayed on the dashboard.



5. If a new Group is added in the Flowsource then the following entry must be made in the entity-yamls -> org.yaml file. Adding the entry will make this group entity available in

flowsource. It will remove the warning entity not found for the group name from the component screen

```
---
# https://backstage.io/docs/features/software-catalog/descriptor-format#kind-group
apiVersion: backstage.io/v1alpha1
kind: Group
metadata:
  name: catalog-user
spec:
  type: team
  children: []
```

12. Flowsource Data Engine installation

Refer *Flowsource-Dataengine-Deployment.docx* for details regarding installation of Data Engine components at ***FlowSource-Package_<realease-version>.zip/data-engine/aws/Flowsource-Dataengine-AWS-Deployment.docx & FlowSource-Package_<realease-version>.zip/data-engine/azure/Flowsource-Dataengine-azure-Deployment.docx***

13. FlowSource CSS changes

Copy below files –

1. /packages-core/app/public/catalog-banner.png
2. /packages-core/app/src/themes/mytheme.js
3. /packages-core/app/src/themes/headerBanner.png

Make changes (highlighted in yellow) to below files –

1. packages-core/app/src/App.tsx –

```
import { myTheme } from './themes/mytheme';
import { UnifiedThemeProvider } from '@backstage/theme';


const app = createApp({
  apis,
  themes: [
    {
      id: 'flowsource-theme',
```

```
      title: 'Flowsource Theme',
      variant: 'light',
      Provider: ({ children }) => (
        <UnifiedThemeProvider theme={FlowsourceTheme} children={children} />
      ),
    },
  ],
```

2. packages-core/app/src/components/Root/Root.tsx

```
const useSidebarStyles = makeStyles({
  sb1: {
    backgroundColor: '#000048 !important',
    color: 'white !important',
    height: '100vh !important',
    width: '100% !important'
  },
});
```

Reference the above “style” code in below declaration (use a comparison tool for this file for identifying differences)

```
export const Root = ({ children }: PropsWithChildren<{}>) =>  {
```

14. Changes required to implement Proxied Authentication using AWS ALB with Azure AD provider

Refer the document for configuration and Flowsource code changes required for implementation of Proxied Authentication using AWS ALB with Azure AD:

FlowSource-Package_<Release-version>/proxied-authentication/AWSALB-AzureAD.pdf

15. Changes required to implement Proxied Authentication using Oauth2 Proxy (with GitHub provider / Azure AD provider)

Refer the document for configuration and Flowsource code changes required for implementation of Proxied Authentication using Outh2Proxy with Azure AD:

FlowSource-Package_<Release-version>/proxied-authentication/OAuth2Proxy-AzureAD.pdf

16. Registering Templates with Flowsource

Templates are scripts, that will generate the boiler plate code needed to help the user with kick starting implementation or setting up an environment, pipeline etc.

A template folder is made up of two parts, the skeleton and template.yaml file,

- The skeleton folder will contain a basic boiler plate code of the template.
- The template.yaml file contains info about fields that should be prompted in FlowSource UI forms, for generating boiler plate code.

These templates can be found in the create tab of Flowsource. The following are the steps to register templates with Flowsource:-

1. A templates folder(FlowSource-templates) is provided in the installer package. This templates folder contains all the template scripts.

   We need to create a SCM repository and upload all the contents of the template folder to the SCM repository.

2. Each required template needs to be referenced in FlowSourceInstaller/terraform-scripts/value-files/all-configurations.yaml file as shown below -

```
allconfigurations:
  files:
    - name: all-configurations.yaml
      content: |-
        apiVersion: backstage.io/v1alpha1
        kind: Location
        metadata:
          name: all-configurations
          description: A collection of all Backstage entities
        spec:
          targets:
            - ./org.yaml
            - ./systems.yaml
            - ./remote-templates.yaml
    - name: remote-tenmplates.yaml
      content: |-
        ---
        apiVersion: backstage.io/v1alpha1
        kind: Location
        metadata:
          name: templates-remote
          description: A collection of remotely available templates
        spec:
          type: url
          targets:
          - https://<url to scm code repository from step 1 above>
```

After following above steps, continue with deployment of FlowSource into K8s cluster by following steps mentioned in the ReadME.md file at FlowSource-Package_<Release-version>/FlowSourceInstaller/ReadMe.MD

17. Proxied Authentication using GCP-IAP with Microsoft Entra ID (Azure Active Directory)

Refer the document for GCP-IAP setup with Azure AD:

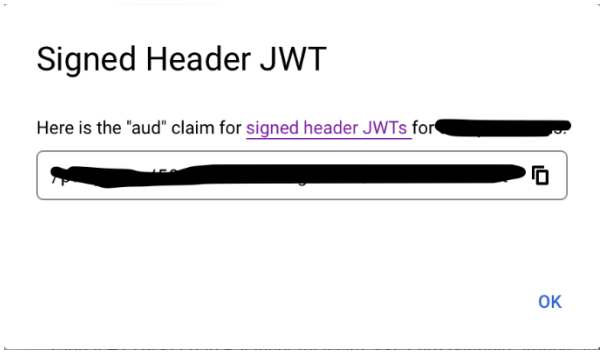
FlowSource-Package_<Release-version>/proxied-authentication/Google-IAP.pdf

Code & Configuration changes:

app-config.yaml

```
----------------------
        auth:
          providers:
            gcp-iap:
              audience: ${GCP_IAP_AUDIENCE}
              signIn:
                resolvers:
                  - resolver: emailMatchingUserEntityProfileEmail
                  - resolver: emailLocalPartMatchingUserEntityName
                  - resolver: emailMatchingUserEntityAnnotation
---------------------------------------------
```

GCP_IAP_AUDIENCE - The full audience value can be obtained by visiting your [Identity-Aware Proxy Google Cloud console](), selecting your project, finding your Backend Service to proxy, clicking the 3 vertical dots then "Get JWT Audience Code", and copying from the resulting popup.



Backend

To add the provider to the backend we will first need to install the package by running this command:

from your Backstage root directory

yarn --cwd packages/backend add @backstage/plugin-auth-backend-module-gcp-iap-provider

Then we will need to add these in:

in **packages-core\backend\src\plugins\auth.ts**

```
import { GcpIapTokenInfo } from '@backstage/plugin-auth-backend-module-gcp-iap-provider';
```

```
function gcpIapAuthHandler(iapToken: GcpIapTokenInfo) {
  const gcpObj = JSON.stringify(iapToken.gcip);
  const parsedObj = JSON.parse(gcpObj);

  const displayName: string | undefined =
    parsedObj.name ?? parsedObj.email ?? parsedObj.user_id;
  const parsedEmail: string = parsedObj.email ?? '';

  return {
    profile: {
      parsedEmail,
      displayName,
    },
  };
}

async function gcpIapResolver(iapToken: GcpIapTokenInfo, database: DatabaseService, ctx: AuthResolverContext) {
  const gcpObj = JSON.stringify(iapToken.gcip);
  const parsedObj = JSON.parse(gcpObj);
  const parsedEmail: string = parsedObj.email ?? '';
  const id = parsedEmail;

  const sub = stringifyEntityRef({
    kind: 'User',
    name: id,
    namespace: DEFAULT_NAMESPACE,
  });

  const db: Knex = await database.getClient();
  const emailToRoleMappingDatabaseService = new EmailToRoleMappingDatabaseService(db);
  const roleMappingDatabaseService = new RoleMappingDatabaseService(db);

  // Get the roles for the user
  let roleArray = await getUserRoles(
    parsedEmail,
    emailToRoleMappingDatabaseService,
  );
```

```
  if (!roleArray || roleArray.length === 0) {
    // If no roles are found for the user, log a warning and continue with the default role
    console.warn('Unauthorized - No roles found for the user - ' + parsedEmail);
  }

  const userRefs = await getUpdatedUserRefs(
    roleArray,
    'gcp-iap',
    roleMappingDatabaseService,
  );

  const ent = userRefs;

  return ctx.issueToken({ claims: { sub, ent } });
}

export const customAuthProvidersModule = createBackendModule({
  // This ID must be exactly "auth" because that's the plugin it targets
  pluginId: 'auth',
  // This ID must be unique, but can be anything
  moduleId: 'custom-auth-providers-module',

  register(reg) {

    reg.registerInit({
      deps: {
        providers: authProvidersExtensionPoint,
        config: coreServices.rootConfig,
        database: coreServices.database,
        logger: coreServices.logger,
      },

      async init({ providers: authProviders, config, database, logger }) {

        // Initialize/create the database
        await initDatabase({
          logger: logger,
          database: database,
        });
        authProviders.registerProvider({
          // This ID must match the actual provider config, e.g. addressing
          // auth.providers.github means that this must be "github".
          providerId: 'gcp-iap',
          // Use createProxyAuthProviderFactory instead if it's one of the proxy
          // based providers rather than an OAuth based one
          factory: providers.gcpIap.create({ // Factory creation based on the old style itself, to
include authHandler logic
            async authHandler({ iapToken }) {
              return gcpIapAuthHandler(iapToken);
            },
```

```
                signIn: {
                  async resolver({ result: { iapToken } }, ctx) {
                    return await gcpIapResolver(iapToken, database, ctx);
                  },
                },
              }),
            });
          },
        });
      },
    });
```

Add in **packages-core\backend\src\index.ts**

```
import { customAuthProvidersModule } from './plugins/auth';

backend.add(import('@backstage/plugin-auth-backend'));
backend.add(customAuthProvidersModule);
```

Frontend

Add in **packages/app/src/App.tsx**

```
const app = createApp({
  components: {
    SignInPage: props => <ProxiedSignInPage {...props} provider="gcp-iap" />

  },
  // ..
});
```

---

18. Rest API to add/ delete rows in role_mappings and email_to_role_mappings in auth database.

A Rest Api is provided to add and delete rows from the role_mappings and email_to_role_mappings tables of backstage_plugin_auth database.

These tables are created during the startup of the application. For both the tables columns ID (auto generated primary key) and created_time (current timestamp) are added.

The structure of the tables would be as follows

```
backstage_plugin_auth=# \d role_mappings
                                        Table "public.role_mappings"
       Column       |           Type           | Collation | Nullable |                  Default
--------------------+--------------------------+-----------+----------+-------------------------------------------
 flowsource_role    | character varying(255)   |           | not null |
 auth_provider      | character varying(255)   |           | not null |
 auth_provider_role | character varying(255)   |           | not null |
 ID                 | integer                  |           | not null | nextval('"role_mappings_ID_seq"'::regclass)
 created_date       | timestamp with time zone |           | not null | CURRENT_TIMESTAMP
Indexes:
    "role_mappings_pkey" PRIMARY KEY, btree ("ID")
    "role_mappings_auth_provider_auth_provider_role_flowsource_role_" UNIQUE CONSTRAINT, btree (auth_provider, auth_provider_role, flowsource_role)
```

```
backstage_plugin_auth=# \d email_to_role_mappings
                                        Table "public.email_to_role_mappings"
       Column       |           Type           | Collation | Nullable |                  Default
--------------------+--------------------------+-----------+----------+-------------------------------------------
 email              | character varying(255)   |           | not null |
 auth_provider_role | character varying(255)   |           | not null |
 ID                 | integer                  |           | not null | nextval('"email_to_role_mappings_ID_seq"'::regclass)
 created_date       | timestamp with time zone |           | not null | CURRENT_TIMESTAMP
Indexes:
    "email_to_role_mappings_pkey" PRIMARY KEY, btree ("ID")
    "email_to_role_mappings_email_auth_provider_role_unique" UNIQUE CONSTRAINT, btree (email, auth_provider_role)
```

Below endpoints and request data remains same for which Authorization Bearer with ROLE_RESTAPI_APIKEY should be sent in request header.

1) PUT: ${backend.baseUrl}/flowsource-core/roleMapping

   body:

```
{
    "flowsourceRole":"catalog-admin",
    "authProvider":"github",
    "authProviderRole":"skygrade-team"
}
```

2) DELETE: ${backend.baseUrl}/flowsource-core/roleMapping
   body:

```
{
    "flowsourceRole":"catalog-admin",
    "authProvider":"github",
    "authProviderRole":"skygrade-team"
}
```

3) PUT: ${backend.baseUrl}/flowsource-core/emailtoRoleMapping
   Body:

```
{
    "email":"abc@xxx.com",
    "authProviderRole":"skygrade-team"
}
```

4) DELETE: ${backend.baseUrl}/flowsource-core/emailtoRoleMapping
Body:

```
{
    "email":"abc@xxx.com",
    "authProviderRole":"skygrade-team"
}
```

1.For role rest api , generate a secret using the below command
node -p 'require("crypto").randomBytes(24).toString("base64")'

2. Update the ROLE_RESTAPI_APIKEY environment variable with the output of the above ommand.

```
externalAccess:
  - type: static
    options:
      token: ${ROLE_RESTAPI_APIKEY}
```

19. AWS Master pipeline template

Introduction

This AWS Master Pipeline Template provides a comprehensive solution for setting up a pipeline using AWS CodeBuild and CodePipeline. With this template, you can automate the build, test, and deployment processes for your applications, ensuring faster and more reliable software delivery.

By leveraging AWS CodeBuild, you can easily build and package your application code, run tests, and generate artifacts. CodePipeline then orchestrates the entire release process, allowing you to define the stages and actions required to deploy your application to various environments.

This guide will walk you through the steps to set up the AWS CodeBuild project and configure the CodePipeline to automate your software delivery pipeline. Before you begin, make sure you have an AWS account, AWS CLI installed and configured, and the necessary permissions to create CodeBuild projects.

Let's get started!

AWS CodeBuild & CodePipeline Project Setup

This guide will walk you through the steps to create a CodeBuild project in AWS.

Prerequisites

Before you begin, ensure you have the following:

- An AWS account
- AWS CLI installed and configured
- Necessary permissions to create CodeBuild projects & CodePipelines.

Steps

1. Create a CodeBuild Project

1. Sign in to the AWS Management Console and open the CodeBuild console at AWS CodeBuild Console.
2. Choose "Create build project".
3. Configure the project:
 - Project name: Enter a unique name for your project.
 - Source provider: This is where you stored your input artifacts for your pipeline. Choose the provider and then provide the connection details. (e.g., CodeCommit, GitHub, S3).
 - Choose the repository & branch name where you have pushed your source code.
 - Environment: Choose the environment image and compute type. You can use a managed image or a custom image.
 - Service Role: Under the service role, select "Create a service role in your account" or "Choose an existing service role" option. There are few permissions needed to create a service role.
 Ex:-

CodeBuild-Service-Role-Policy

```
{
                "Version": "2012-10-17",
                "Statement": [
                    {
                        "Sid": "AssumeRole",
                        "Effect": "Allow",
                        "Action": "sts:AssumeRole",
                        "Resource":
"arn:aws:iam::${AWS::AccountId}:role/codebuild-service-role"
                    },
                    {
                        "Sid": "EKS",
                        "Effect": "Allow",
                        "Action": [
                            "eks:Describe*"
                        ],
                        "Resource": "*"
                    },
                    {
                        "Sid": "ECR",
```

```
                        "Effect": "Allow",
                        "Action": [
                            "ecr:GetAuthorizationToken",
                            "ecr:BatchCheckLayerAvailability",
                            "ecr:BatchGetImage",
                            "ecr:GetDownloadUrlForLayer",
                            "ecr:DescribeRepositories",
                            "ecr:ListImages",
                            "ecr:DescribeImages",
                            "ecr:DescribeImageScanFindings",
                            "ecr:GetRepositoryPolicy",
                            "ecr:GetLifecyclePolicy",
                            "ecr:GetLifecyclePolicyPreview",
                            "ecr:ListTagsForResource",
                            "ecr:PutImage",
                            "ecr:InitiateLayerUpload",
                            "ecr:UploadLayerPart",
                            "ecr:CompleteLayerUpload",
                            "ecr:PutLifecyclePolicy",
                            "ecr-public:*"
                        ],
                        "Resource": "arn:aws:ecr:us-east-
1:${AWS::AccountId}:repository/demo-aws-ecr"
                    },
                    {
                        "Sid": "ECRAuthorizationToken",
                        "Effect": "Allow",
                        "Action": "ecr:GetAuthorizationToken",
                        "Resource": "*"
                    },
                    {
                        "Sid": "CloudWatch",
                        "Effect": "Allow",
                        "Action": [
                            "logs:CreateLogGroup",
                            "logs:CreateLogStream",
                            "logs:PutLogEvents"
                        ],
                        "Resource": [
                            "arn:aws:logs:us-east-
1:${AWS::AccountId}:log-group:aws/codebuild/*",
                            "arn:aws:logs:us-east-
1:${AWS::AccountId}:log-group:/aws/codebuild/*:log-stream:*"
                        ]
                    },
                    {
```

```
                        "Sid": "SecretsManager",
                        "Effect": "Allow",
                        "Action": [
                            "secretsmanager:GetSecretValue",
                            "secretsmanager:DescribeSecret"
                        ],
                        "Resource": "arn:aws:secretsmanager:us-east-
1:${AWS::AccountId}:secret:*"
                    },
                    {
                        "Sid": "CodeBuild",
                        "Effect": "Allow",
                        "Action": [
                            "codebuild:CreateReportGroup",
                            "codebuild:CreateReport",
                            "codebuild:DeleteReportGroup",
                            "codebuild:UpdateReportGroup",
                            "codebuild:BatchGetReportGroups",
                            "codebuild:ListReportGroups",
                            "codebuild:BatchGetReports",
                            "codebuild:DeleteReport",
                            "codebuild:UpdateReport",
                            "codebuild:ListReports",
                            "codebuild:ListReportsForReportGroup",
                            "codebuild:CreateProject",
                            "codebuild:BatchGetProjects",
                            "codebuild:DeleteProject",
                            "codebuild:ListProjects",
                            "codebuild:UpdateProject",
                            "codebuild:StartBuild",
                            "codebuild:BatchGetBuilds",
                            "codebuild:ListBuilds",
                            "codebuild:StopBuild",
                            "iam:PassRole"
                        ],
                        "Resource": "arn:aws:codebuild:us-east-
1:${AWS::AccountId}:project/*"
                    },
                    {
                        "Sid": "CodePipeline",
                        "Effect": "Allow",
                        "Action": [
                            "codepipeline:CreatePipeline",
                            "codepipeline:ListPipelines",
                            "codepipeline:GetPipeline",
                            "codepipeline:UpdatePipeline",
```

```
                            "codepipeline:DeletePipeline",
                            "codepipeline:ListTagsForResource"
                        ],
                        "Resource": "arn:aws:codepipeline:us-east-
1:${AWS::AccountId}:*"
                    },
                    {
                        "Sid": "S3CloudBuild",
                        "Effect": "Allow",
                        "Action": [
                            "s3:PutObject",
                            "s3:GetObject",
                            "s3:ListBucket"
                        ],
                        "Resource": "*"
                    },
                    {
                        "Sid": "Terraform",
                        "Effect": "Allow",
                        "Action": [
                            "eks:*",
                            "ec2:DescribeInstances",
                            "ec2:DescribeNetworkInterfaces",
                            "ec2:DescribeRouteTables",
                            "ec2:DescribeSecurityGroups",
                            "ec2:DescribeSubnets",
                            "ec2:DescribeVpcs",
                            "ec2:CreateSecurityGroup",
                            "ec2:AuthorizeSecurityGroupIngress",
                            "ec2:RevokeSecurityGroupIngress",
                            "ec2:AuthorizeSecurityGroupEgress",
                            "ec2:RevokeSecurityGroupEgress",
                            "ec2:DeleteSecurityGroup",
                            "elasticloadbalancing:*",
                            "iam:CreateRole",
                            "iam:PutRolePolicy",
                            "iam:AttachRolePolicy",
                            "iam:DetachRolePolicy",
                            "iam:DeleteRolePolicy",
                            "iam:DeleteRole",
                            "autoscaling:*",
                            "cloudwatch:*",
                            "logs:*",
                            "ssm:*",
                            "ec2:DeleteNetworkInterface",
                            "ec2:CreateNetworkInterface",
```

```
                            "ec2:CreateNetworkInterfacePermission",
                            "ec2:DescribeDhcpOptions"
                        ],
                        "Resource": "*"
                    },
                    {
                        "Sid": "DynamoDB",
                        "Effect": "Allow",
                        "Action": [
                            "dynamodb:GetItem",
                            "dynamodb:PutItem",
                            "dynamodb:DeleteItem"
                        ],
                        "Resource": "*"
                    },
                    {
                        "Sid": "IAMCreatePolicy",
                        "Effect": "Allow",
                        "Action": [
                            "iam:CreatePolicy",
                            "iam:TagPolicy"
                        ],
                        "Resource": "*"
                    },
                    {
                        "Effect": "Allow",
                        "Action": [
                            "ec2:CreateNetworkInterfacePermission",
                            "ec2:DescribeDhcpOptions",
                            "ec2:DescribeNetworkInterfaces",
                            "ec2:DeleteNetworkInterface",
                            "ec2:DescribeSubnets",
                            "ec2:DescribeSecurityGroups",
                            "ec2:DescribeVpcs"
                        ],
                        "Resource": "arn:aws:ec2:us-east-
1:${AWS::AccountId}:network-interface/*",
                        "Condition": {
                            "StringEquals": {
                                "ec2:AuthorizedService":
"codebuild.amazonaws.com"
                            },
                            "ArnEquals": {
                                "ec2:Subnet": [
                                    "arn:aws:ec2:us-east-
1:${AWS::AccountId}:subnet/subnet-046b20ce41dbdd427",
```

```
                                        "arn:aws:ec2:us-east-
1:${AWS::AccountId}:subnet/subnet-004d6ab500ba9634c",
                                        "arn:aws:ec2:us-east-
1:${AWS::AccountId}:subnet/subnet-0445fdb7cf2294dbe",
                                        "arn:aws:ec2:us-east-
1:${AWS::AccountId}:subnet/subnet-0984629ab6f1058d3"
                                    ]
                                }
                            }
                        }
                    ]
                }
```

CodeBuild-Service-Role-Trust-Relationship

```
{
        "Version": "2012-10-17",
        "Statement": [
            {
                "Effect": "Allow",
                "Principal": {
                    "AWS": [
                        "arn:aws:iam::${AWS::AccountId}:root"
                    ]
                },
                "Action": "sts:AssumeRole"
            },
            {
                "Effect": "Allow",
                "Principal": {
                    "Service": [
                        "eks.amazonaws.com",
                        "codebuild.amazonaws.com"
                    ]
                },
                "Action": "sts:AssumeRole"
            }
        ]
    }
```

Note - Replace `${AWS::AccountId}` & `${AWS::Region}` with your own accountId & region.

- **VPC & SubNet**: Under additional configuration add the VPC and subnet for your CodeBuild project, select the desired VPC from the dropdown menu. Choose the appropriate subnets from the available options. If needed, specify additional network settings such as security groups and validate VPC settings.
- **Environment Variables**: Under the "Environment" section specify the environment variables by clicking on "Add environment variable". Enter the variable name and its value.

Ex:- CodeBuild Variables

Name	Value	Type
REGION	#{variables.REGION}	PLAINTEXT
PIPELINE_NAME	#{variables.PIPELINE_NAME}	PLAINTEXT
PROJECT_NAME	#{variables.PROJECT_NAME}	PLAINTEXT
REPOSITORY_NAME	#{variables.REPOSITORY_NAME}	PLAINTEXT
BRANCH_NAME	#{variables.BRANCH_NAME}	PLAINTEXT
BUILDSPEC	#{variables.BUILDSPEC}	PLAINTEXT
USE_VPC	#{variables.USE_VPC}	PLAINTEXT
VPC_ID	#{variables.VPC_ID}	PLAINTEXT
SUBNET1	#{variables.SUBNET1}	PLAINTEXT
SUBNET2	#{variables.SUBNET2}	PLAINTEXT
SECURITY_GROUP_ID	#{variables.SECURITY_GROUP_ID}	PLAINTEXT

- **Buildspec**: Specify the build commands in a `buildspec.yml` file. This file should be in the root of your source code repository.
- **Artifacts**: Define where the build output should be stored (e.g., S3 bucket).
- **Logs**: Optionally, configure logging to Amazon CloudWatch.

4. **Click** "Create build project" to save your settings.

2. Configure AWS CodePipeline

1. Sign in to the AWS Management Console and open the CodePipeline console at AWS CodePipeline Console.
2. Choose "Create pipeline".
3. Configure the pipeline:
   - Pipeline name: Enter a unique name for your pipeline.
   - Exection mode: Choose the execution mode for your pipeline. This determines how the

pipeline is run.

- Service Role: Under the service role, select "Create a service role in your account" or "Choose an existing service role" option. There are permissions needed to create a service role.

Ex:-

CodePipeline-Service-Role-Policy

```
{
"Version": "2012-10-17",
"Statement": [
    {
        "Sid": "AssumeRole",
        "Effect": "Allow",
        "Action": "sts:AssumeRole",
        "Resource":
"arn:aws:iam::${AWS::AccountId}:role/codebuild-service-role"
    },
    {
        "Action": [
            "iam:PassRole"
        ],
        "Resource": "*",
        "Effect": "Allow",
        "Condition": {
            "StringEqualsIfExists": {
                "iam:PassedToService": [
                    "cloudformation.amazonaws.com",
                    "elasticbeanstalk.amazonaws.com",
                    "ec2.amazonaws.com",
                    "ecs-tasks.amazonaws.com"
                ]
            }
        }
    },
    {
        "Action": [
            "codecommit:CancelUploadArchive",
            "codecommit:GetBranch",
            "codecommit:GetCommit",
            "codecommit:GetRepository",
            "codecommit:GetUploadArchiveStatus",
            "codecommit:UploadArchive"
        ],
```

```
                "Resource": "*",
                "Effect": "Allow"
            },
            {
                "Action": [
                    "codedeploy:CreateDeployment",
                    "codedeploy:GetApplication",
                    "codedeploy:GetApplicationRevision",
                    "codedeploy:GetDeployment",
                    "codedeploy:GetDeploymentConfig",
                    "codedeploy:RegisterApplicationRevision"
                ],
                "Resource": "*",
                "Effect": "Allow"
            },
            {
                "Action": [
                    "codestar-connections:UseConnection"
                ],
                "Resource": "*",
                "Effect": "Allow"
            },
            {
                "Action": [
                    "elasticbeanstalk:*",
                    "ec2:*",
                    "elasticloadbalancing:*",
                    "autoscaling:*",
                    "cloudwatch:*",
                    "s3:*",
                    "sns:*",
                    "cloudformation:*",
                    "rds:*",
                    "sqs:*",
                    "ecs:*"
                ],
                "Resource": "*",
                "Effect": "Allow"
            },
            {
                "Action": [
                    "lambda:InvokeFunction",
                    "lambda:ListFunctions"
                ],
                "Resource": "*",
                "Effect": "Allow"
```

```
            },
            {
                "Action": [
                    "opsworks:CreateDeployment",
                    "opsworks:DescribeApps",
                    "opsworks:DescribeCommands",
                    "opsworks:DescribeDeployments",
                    "opsworks:DescribeInstances",
                    "opsworks:DescribeStacks",
                    "opsworks:UpdateApp",
                    "opsworks:UpdateStack"
                ],
                "Resource": "*",
                "Effect": "Allow"
            },
            {
                "Action": [
                    "cloudformation:CreateStack",
                    "cloudformation:DeleteStack",
                    "cloudformation:DescribeStacks",
                    "cloudformation:UpdateStack",
                    "cloudformation:CreateChangeSet",
                    "cloudformation:DeleteChangeSet",
                    "cloudformation:DescribeChangeSet",
                    "cloudformation:ExecuteChangeSet",
                    "cloudformation:SetStackPolicy",
                    "cloudformation:ValidateTemplate"
                ],
                "Resource": "*",
                "Effect": "Allow"
            },
            {
                "Action": [
                    "codebuild:BatchGetBuilds",
                    "codebuild:StartBuild",
                    "codebuild:BatchGetBuildBatches",
                    "codebuild:StartBuildBatch"
                ],
                "Resource": "*",
                "Effect": "Allow"
            },
            {
                "Effect": "Allow",
                "Action": [
                    "devicefarm:ListProjects",
                    "devicefarm:ListDevicePools",
```

```
                        "devicefarm:GetRun",
                        "devicefarm:GetUpload",
                        "devicefarm:CreateUpload",
                        "devicefarm:ScheduleRun"
                    ],
                    "Resource": "*"
                },
                {
                    "Effect": "Allow",
                    "Action": [
                        "servicecatalog:ListProvisioningArtifacts",
                        "servicecatalog:CreateProvisioningArtifact",

"servicecatalog:DescribeProvisioningArtifact",
                        "servicecatalog:DeleteProvisioningArtifact",
                        "servicecatalog:UpdateProduct",
                        "ec2:DeleteNetworkInterface"
                    ],
                    "Resource": "*"
                },
                {
                    "Effect": "Allow",
                    "Action": [
                        "cloudformation:ValidateTemplate"
                    ],
                    "Resource": "*"
                },
                {
                    "Effect": "Allow",
                    "Action": [
                        "ecr:DescribeImages"
                    ],
                    "Resource": "*"
                },
                {
                    "Effect": "Allow",
                    "Action": [
                        "states:DescribeExecution",
                        "states:DescribeStateMachine",
                        "states:StartExecution"
                    ],
                    "Resource": "*"
                },
                {
                    "Effect": "Allow",
                    "Action": [
```

```
                        "appconfig:StartDeployment",
                        "appconfig:StopDeployment",
                        "appconfig:GetDeployment"
                    ],
                    "Resource": "*"
                }
            ]
        }
```

CodePipeline-Service-Role-Trust-Relationship

```
{
        "Version": "2012-10-17",
        "Statement": [
            {
                "Sid": "CodePipelineServiceRole",
                "Effect": "Allow",
                "Principal": {
                    "Service": "codepipeline.amazonaws.com"
                },
                "Action": "sts:AssumeRole"
            }
        ]
    }
```

Note - Replace `${AWS::AccountId}` & `${AWS::Region}` with your own accountId & region.

- **Variables**: You can add variables at the pipeline level. You can choose to assign the value when you start the pipeline. Choosing this option requires pipeline type V2.

Ex:- CodePipeline Variables

Name	Default value	Description	Usage
REGION	us-east-1	Region where you would need to setup pipeline	CI & CD pipelines
PIPELINE_NAME		Name of the CodePipeline you need to create	CI & CD pipelines
PROJECT_NAME		Name of the CodeBuild project you need to create	CI & CD pipelines
REPOSITORY_NAME		Repository for which the pipeline to be created	CI & CD pipelines

BRANCH_NAME		Branch to be used to configure pipeline	CI & CD pipelines
BUILDSPEC		buildspec file path to configure pipeline	CI & CD pipelines
USE_VPC	FALSE	Mark it as True if you need setup a CD pipeline	CD pipeline only
VPC_ID	VPC_ID	Provide VPC ID for the deployment via CD pipeline	CD pipeline only
SUBNET1	SUBNET1	Provide SUBNET1 for the deployment via CD pipeline	CD pipeline only
SUBNET2	SUBNET2	Provide SUBNET2 for the deployment via CD pipeline	CD pipeline only
SECURITY_GROUP_ID	SECURITY_GROUP_ID	Provide SECURITY_GROUP_ID for the deployment via CD pipeline	pipeline only

- Source provider: This is where you stored your input artifacts for your pipeline. Choose the provider and then provide the connection details. (e.g., CodeCommit, GitHub, S3).
- Choose the repository & branch name where you have pushed your source code.
- Build provider: Choose AWS CodeBuild as the build provider. Choose a build project that you have already created in the AWS CodeBuild console. Or create a build project in the AWS CodeBuild console and then return to this task. Specify these environment variables for the build project.

Ex-CodeBuild Variables

Name	Value	Type
REGION	#{variables.REGION}	PLAINTEXT
PIPELINE_NAME	#{variables.PIPELINE_NAME}	PLAINTEXT
PROJECT_NAME	#{variables.PROJECT_NAME}	PLAINTEXT
REPOSITORY_NAME	#{variables.REPOSITORY_NAME}	PLAINTEXT
BRANCH_NAME	#{variables.BRANCH_NAME}	PLAINTEXT
BUILDSPEC	#{variables.BUILDSPEC}	PLAINTEXT
USE_VPC	#{variables.USE_VPC}	PLAINTEXT
VPC_ID	#{variables.VPC_ID}	PLAINTEXT
SUBNET1	#{variables.SUBNET1}	PLAINTEXT

SUBNET2	#{variables.SUBNET2}	PLAINTEXT
SECURITY_GROUP_ID	#{variables.SECURITY_GROUP_ID}	PLAINTEXT

- Deployment provider: Skip this steps as EKS is not supported for CodeDeploy.
- Pipeline Review: Review your selections for the pipeline to be created. The stages and action providers in each stage are shown in the order that they will be created.

4. Click "Create pipeline" to create your pipeline.
5. Trigger the pipeline: You can manually trigger the pipeline or set up a trigger based on changes to your source code repository.
6. Monitor the pipeline: Monitor the progress of your pipeline and view the logs and artifacts generated by each stage.

20. GCP Master Pipeline Template

Introduction

This GCP Master Pipeline Template provides a comprehensive solution for setting up a pipeline using Google Cloud Build and Google Cloud Deploy. With this template, you can automate the build, test, and deployment processes for your applications, ensuring faster and more reliable software delivery.

By leveraging Google Cloud Build, you can easily build and package your application code, run tests, and generate artifacts.

This guide will walk you through the steps to set up the Google Cloud Build project. Before you begin, make sure you have a Google Cloud account, Google Cloud SDK installed and configured, and the necessary permissions to create Cloud Build projects.

Let’s get started!

Google Cloud Build Project Setup

This guide will walk you through the steps to create a Cloud Build project in GCP.

Prerequisites Before you begin, ensure you have the following:

1. GitHub Connection:

Before running Terraform to create the Master Pipeline (Google Cloud Build Trigger), it is required that a GitHub connection is already configured.

After the GitHub Connection is configured, we also need to link the desired GitHub Repositories to Cloud Build.

To configure the GitHub Connection and link the Repositories with Google Cloud Build, please follow these steps - https://cloud.google.com/build/docs/automating-builds/github/connect-repo-github?generation=1st-gen

Note: In V2, we don't have to manually link the repositories. They are linked by the Master Pipeline's Trigger automatically.

2. Service Account:

A Service Account is required to run the builds on Google Cloud Build. And we need to pass the Service Account Email as an input to Terraform to create the Master Pipeline.

Create a Cloud Build Project

Sign in to the Google Cloud Console and open the Cloud Build console. Choose “Create build trigger”. Project name: Enter a unique name for your project. Region: Enter the region name where you want to create your trigger. Description: Add small and exact description of your trigger. Tags: Add the tags if needed. Events: Add the events on which the trigger get started. Source: Select the source repo and branch for which you want to create the trigger. Configuration: Specify the cloudbuild.yaml file and location which you want to use for build.

Substitution Variables: Under the "Advanced" section specify the variables by clicking on "Add variable". Enter the variable name and its value. Ex:-

CloudBuild Variables

Name	Value
_BRANCH_NAME	
_CLOUDBUILD_YAML_PATH	
_GITHUB_CONNECTION_NAME	
_GITHUB_REPO_NAME	
_GITHUB_REPO_OWNER	
_REGION	
_SERVICE_ACCOUNT_EMAIL	
_TRIGGER_NAME	

Approval: If you want a check before build executes. you can check this option. Logs: If you want to send logs to github. We can check this option

Service Account: Now select the service account with necessary permission to trgger the build.

Click on Create to create the trigger.

Trigger the pipeline: You can manually trigger the build or set up a trigger based on changes to your source code repository. Monitor the pipeline: Monitor the progress of your build and view the logs and artifacts generated by each stage.

21. Azure DevOps Discovery

The Azure DevOps integration has a special entity provider for discovering catalog entities within an Azure DevOps. The provider will crawl your Azure DevOps organization and register entities matching the configured path. This can be useful as an alternative to static locations or manually adding things to the catalog.

This is implemented as per backstage guidelines. Please refer to https://backstage.io/docs/integrations/azure/discovery/ for detailed instructions.